

Regresja liniowa od podstaw

Matematyka, implementacja i analiza projektu

Kompletny przewodnik do pierwszego projektu machine learningowego: model liniowy, metoda najmniejszych kwadratów, gradient descent, mini-batche, Momentum, Adam, regularyzacja Ridge, metryki oraz interpretacja statystyczna. Dokument łączy formalne wyprowadzenia matematyczne z dokładną analizą implementacji w Pythonie i NumPy.

Spis treści

Jak korzystać z tego dokumentu	5
1. Obraz całego projektu	5
1.1. Jaki problem rozwiązuje program?	5
1.2. Przepływ informacji	5
1.3. Odpowiedzialność poszczególnych plików	5
2. Notacja, dane i wymiary	6
3. Czym jest regresja liniowa	6
3.1. Model dla jednej cechy	6
3.2. Model dla wielu cech	7
3.3. Parametry a hiperparametry	7
4. Dane syntetyczne w projekcie	7
4.1. Macierz cech	7
4.2. Prawdziwe współczynniki	8
4.3. Szum	8
4.4. Brak pełnej powtarzalności	8
5. Funkcja kosztu MSE	8
5.1. Od reszty do funkcji celu	8
5.2. Postać macierzowa	9
5.3. Dlaczego kwadrat?	9
5.4. Interpretacja probabilistyczna	9
5.5. MSE a czynnik $1/2$	10
6. Gradient MSE - pełne wyprowadzenie	10
6.1. Co oznacza gradient?	10
6.2. Wyprowadzenie skalarne	10
6.3. Wyprowadzenie macierzowe	11
6.4. Przełożenie na kod	11
7. Geometria, wypukłość i Hessian	12
7.1. Powierzchnia funkcji kosztu	12
7.2. Hessian	12
7.3. Wartości własne i learning rate	12
7.4. Uwarunkowanie i skalowanie cech	13
8. Rozwiązanie analityczne i <code>np.linalg.lstsq</code>	13
8.1. Włączenie biasu do macierzy	13
8.2. Normal equations	13
8.3. Dlaczego kod nie oblicza odwrotności?	14
8.4. Rozpakowanie wyniku	14
8.5. Closed form kontra gradient descent	14
9. Gradient descent	15
9.1. Podstawowa reguła	15
9.2. Co może się wydarzyć podczas treningu?	15
9.3. Inicjalizacja zerami	15
9.4. Epoka nie jest tym samym co krok	16
10. Batch GD, mini-batch GD i SGD	16
10.1. Full-batch gradient descent	16
10.2. Mini-batch gradient descent	16
10.3. Stochastic gradient descent	16

10.4. Slicing batchy	17
10.5. Dlaczego tasowanie jest ważne?	17
10.6. Subtelność <code>loss_history</code>	17
11. Implementacja klasy <code>LinearRegression</code>	17
11.1. Konstruktor	17
11.2. <code>fit</code>	18
11.3. <code>predict</code>	19
11.4. <code>score</code>	19
11.5. Inicjalizacja stanu optymalizatorów	19
11.6. Pętla treningowa	19
11.7. Historia wag i <code>.copy()</code>	20
12. Vanilla GD, Momentum i Adam	20
12.1. Wspólna idea optymalizatora	20
12.2. Vanilla gradient descent	20
12.3. Momentum	20
12.4. Adam	21
12.5. Typy skalarne i tablicowe	23
13. Ridge i współliniowość	23
13.1. Konstrukcja cechy prawie zduplikowanej	23
13.2. Dlaczego współczynniki stają się niestabilne?	23
13.3. Kara Ridge	23
13.4. Wpływ α	24
13.5. Rozwiązanie analityczne Ridge	24
13.6. Ridge nie jest Lasso	24
13.7. Interpretacja bayesowska	24
14. Metryki MSE i R^2	25
14.1. MSE jako loss i metryka	25
14.2. Współczynnik determinacji	25
14.3. Interpretacja	25
14.4. Przypadek stałego targetu	25
14.5. Ocena treningowa a generalizacja	26
15. Eksperymenty z <code>run_experiments.py</code>	26
15.1. Importy	26
15.2. Test rozwiązania analitycznego	26
15.3. Test gradient descent	26
15.4. Full-batch kontra mini-batch	27
15.5. Eksperyment ze współliniowością i Ridge	27
15.6. Historia wag	27
15.7. California Housing	28
15.8. Funkcja testowa dla optymalizatorów	28
15.9. Zaszumiony gradient	29
15.10. Vanilla na funkcji testowej	29
15.11. Momentum na funkcji testowej	29
15.12. Adam na funkcji testowej	30
15.13. Czy porównanie jest uczciwe?	30
15.14. Punkt wejścia programu	30
16. Wizualizacje	31
16.1. Katalog wynikowy	31
16.2. <code>fig</code> i <code>ax</code>	31
16.3. Krzywe loss	31
16.4. Predykcje względem prawdy	31

16.5. Zbieżność wag	32
16.6. Porównanie optymalizatorów	32
16.7. Zapisywanie i zarządzanie figurami	32
17. Python, NumPy i struktura pakietu	32
17.1. Pusty <code>__init__.py</code>	32
17.2. Adnotacje typów	33
17.3. Operacje wektorowe zamiast pętli	33
17.4. Operacje elementowe a macierzowe	33
17.5. <code>np.hstack</code> i zachowanie wymiaru	33
17.6. Przypisanie wielu wartości	34
17.7. Stan modelu i stan optymalizatora	34
18. Założenia regresji i interpretacja statystyczna	34
18.1. Liniowość w parametrach	34
18.2. Egzogeniczność	34
18.3. Brak doskonałej współliniowości	34
18.4. Homoskedastyczność	34
18.5. Niezależność lub odpowiednia struktura zależności	35
18.6. Normalność błędów	35
18.7. Gauss-Markov	35
18.8. Średnia i wariancja estymatora OLS	35
18.9. OLS jako projekcja	36
19. Audyt projektu i miejsca do poprawy	36
19.1. Co jest zrobione dobrze?	36
19.2. Rzeczy wpływające na interpretację wyników	36
19.3. Brak walidacji wejścia	37
19.4. Nieznana metoda lub optimizer	37
19.5. Predykcja przed treningiem	37
19.6. <code>seed</code> nie steruje losowością	37
19.7. Ponowne <code>fit</code>	37
19.8. <code>alpha</code> i metoda closed form	37
19.9. Test Ridge przez normę L1	37
19.10. Adnotacje typów optimizerów	38
19.11. Szczególny przypadek R^2	38
19.12. Zarządzanie wykresami	38
19.13. Stopień ważności uwag	38
20. Ręczny przykład liczbowy	38
20.1. Predykcja	38
20.2. Reszty i MSE	38
20.3. <code>d_out</code>	39
20.4. Gradient wag	39
20.5. Gradient biasu	39
20.6. Jeden krok Vanilla GD	39
20.7. Wpływ Ridge w tym samym punkcie	39
20.8. Pierwszy krok Momentum	40
20.9. Pierwszy krok Adama	40
20.10. Przykład jednego parametru Adama przez dwa kroki	40
21. Połączenie z dalszym machine learningiem	41
21.1. Regresja jako pojedynczy neuron liniowy	41
21.2. <code>d_out</code> jako początek backpropagation	41
21.3. Stan optymalizatora skaluje się do sieci	41
21.4. Mini-batche stają się koniecznością	41
21.5. Różnica: problem wypukły i niewypukły	42

21.6. Regularizacja	42
22. Pytania kontrolne i eksperymenty	42
22.1. Pytania koncepcyjne	42
22.2. Ćwiczenia matematyczne	42
22.3. Eksperymenty kodowe	43
22.4. Kryterium pełnego zrozumienia projektu	43
23. Słownik pojęć	44
Aneks A. Niezmieniony kod analizowany w dokumencie	45
A.1. <code>metrics.py</code>	45
A.2. <code>optimizers.py</code>	46
A.3. <code>model.py</code>	47
A.4. <code>run_experiments.py</code>	48
A.5. <code>visualize.py</code>	50
A.6. <code>__init__.py</code>	51
Końcowa synteza	52

Jak korzystać z tego dokumentu

Ten tekst ma dwa cele. Po pierwsze, ma wyjaśnić **dokładnie ten projekt**, a nie jedynie abstrakcyjną regresję liniową. Po drugie, ma zbudować podstawy potrzebne później w sieciach neuronowych i bardziej zaawansowanym machine learningu.

Przy pierwszym wystąpieniu angielskiego terminu podaję polskie znaczenie i krótkie objaśnienie. Na końcu znajduje się również słownik. Matematyka zakłada znajomość analizy matematycznej i algebry liniowej na poziomie akademickim: nie wyjaśniam od zera mnożenia macierzy, ale dokładnie wyprowadzam funkcję kosztu, gradienty i algorytmy optymalizacji.

Warto czytać rozdziały po kolei. Kod jest prosty, lecz łączy kilka odrębnych poziomów:

1. model statystyczny opisujący zależność między zmiennymi,
2. funkcję celu określającą, co znaczy „dobry model”,
3. algorytm znajdujący parametry modelu,
4. implementację numeryczną w NumPy,
5. eksperymenty sprawdzające zachowanie implementacji.

1. Obraz całego projektu

1.1. Jaki problem rozwiązuje program?

Projekt uczy model przewidywać wartość liczbową na podstawie kilku cech. Taki problem nazywa się **regresją** (*regression*). Przykładowo:

- przewidywanie ceny mieszkania na podstawie powierzchni, lokalizacji i wieku budynku,
- przewidywanie sprzedaży na podstawie ceny i wydatków reklamowych,
- przewidywanie wynagrodzenia na podstawie doświadczenia i wykształcenia.

Model przyjmuje wektor cech obserwacji i zwraca jedną liczbę. Zakłada przy tym liniową postać zależności:

$$\hat{y} = x^\top w + b.$$

Symbol $\hat{y}$ czytamy „y z daszkiem” i oznacza on **predykcję** (*prediction*), czyli wartość przewidzianą przez model. y bez daszka jest wartością rzeczywistą.

1.2. Przepływ informacji

Cały projekt można streścić następująco:

$$(X, y) \xrightarrow{\text{fit}} (w, b) \xrightarrow{\text{predict}} \hat{y} \xrightarrow{\text{metryka}} \text{ocena modelu.}$$

- `fit(X, y)` - **dopasowuje** (*fits/trains*) model, czyli wybiera parametry w i b .
- `predict(X)` - oblicza predykcje dla danych parametrów.
- `score(X, y)` - porównuje predykcje z rzeczywistymi wartościami przez R^2 .

Termin **training** oznacza uczenie modelu, a więc proces poszukiwania parametrów minimalizujących funkcję kosztu. Model nie „rozumie” danych w ludzkim sensie. W tej implementacji trening oznacza wykonywanie kolejnych działań arytmetycznych na tablicach.

1.3. Odpowiedzialność poszczególnych plików

Plik	Odpowiedzialność
<code>metrics.py</code>	Definicje MSE i R^2 , czyli sposobów pomiaru błędu i jakości predykcji.
<code>optimizers.py</code>	Reguły aktualizacji parametrów: Vanilla GD, Momentum i Adam.
<code>model.py</code>	Klasa regresji, trening, predykcja, rozwiązanie analityczne i Ridge.

Plik	Odpowiedzialność
<code>run_experiments.py</code>	Generowanie danych, uruchamianie modeli, asercje i eksperymenty.
<code>visualize.py</code>	Tworzenie i zapisywanie wykresów.
<code>__init__.py</code>	Oznaczenie katalogu jako pakietu Pythona; plik może pozostać pusty.

Rozdzielenie kodu jest sensowne: model nie musi wiedzieć, jak rysuje się wykres, a funkcja optymalizatora nie musi znać pochodzenia gradientu.

2. Notacja, dane i wymiary

Niech:

- n - liczba obserwacji (*samples*),
- d - liczba cech (*features*),
- $X \in \mathbb{R}^{n \times d}$ - macierz cech,
- $x_i^\top \in \mathbb{R}^{1 \times d}$ - i -ty wiersz X , czyli cechy jednej obserwacji,
- $w \in \mathbb{R}^d$ - wektor wag (*weights*),
- $b \in \mathbb{R}$ - wyraz wolny, nazywany również biasem lub interceptem,
- $y \in \mathbb{R}^n$ - wektor wartości rzeczywistych,
- $\hat{y} \in \mathbb{R}^n$ - wektor predykcji,
- $e = \hat{y} - y \in \mathbb{R}^n$ - wektor błędów albo reszt.

W danych syntetycznych z projektu $n = 200$ i $d = 3$.

Obiekt w kodzie	Matematyka	<code>shape</code> w danych syntetycznych
<code>X</code>	X	$(200, 3)$
<code>X[i]</code>	x_i	$(3,)$
<code>self.w</code>	w	$(3,)$
<code>self.b</code>	b	skalar
<code>y</code>	y	$(200,)$
<code>y_pred</code>	$\hat{y}$	$(200,)$ lub rozmiar batcha
<code>d_out</code>	$\partial J / \partial \hat{y}$	jak <code>y_batch</code>
<code>grad_w</code>	$\nabla_w J$	$(3,)$
<code>grad_b</code>	$\partial J / \partial b$	skalar

Ważna cecha NumPy

Jednowymiarowa tablica `np.ndarray` o `shape == (3,)` nie jest formalnie ani macierzą wierszową $(1, 3)$, ani kolumnową $(3, 1)$. NumPy stosuje własne reguły dla operatora `@`. W zapisie matematycznym będziemy traktować w jako wektor kolumnowy, ale w kodzie jest on tablicą jednowymiarową.

Dzięki temu:

```
X @ self.w
```

ma wymiar $(n,)$. Następnie skalar `self.b` jest dodawany do każdego elementu. To automatyczne rozszerzenie skalaru na wszystkie elementy nazywa się **broadcastingiem**.

3. Czym jest regresja liniowa

3.1. Model dla jednej cechy

Dla jednej cechy model ma postać:

$$\hat{y}_i = wx_i + b.$$

Waga w opisuje zmianę przewidywanego y przy wzroście x o jednostkę:

$$\hat{y}(x+1) - \hat{y}(x) = w.$$

Bias b jest predykcją dla $x = 0$. Geometrycznie model jest prostą.

3.2. Model dla wielu cech

Dla d cech:

$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id} + b = x_i^\top w + b.$$

Dla wszystkich obserwacji jednocześnie:

$$\hat{y} = Xw + b\mathbf{1},$$

gdzie $\mathbf{1} \in \mathbb{R}^n$ jest wektorem jedynek.

Nazwa „liniowa” odnosi się przede wszystkim do liniowości względem parametrów. Model może zawierać wcześniej skonstruowane cechy x^2 czy $\log x$ i nadal być liniowy względem współczynników, np.

$$\hat{y} = w_1 x + w_2 x^2 + b.$$

Nie jest to prosta względem x , ale jest to liniowa kombinacja parametrów w_1, w_2, b .

3.3. Parametry a hiperparametry

Parametry (*parameters*) są wyznaczane na podstawie danych:

- `self.w`,
- `self.b`.

Hiperparametry (*hyperparameters*) ustala programista przed treningiem:

- `lr`,
- `n_epochs`,
- `batch_size`,
- `alpha`,
- wybór `optimizer`.

Model uczy się wag, ale sam nie uczy się w tym projekcie learning rate ani liczby epok.

4. Dane syntetyczne w projekcie

Kod tworzy kontrolowany problem, dla którego znamy prawdziwe parametry:

```
X = np.random.randn(200, 3)
true_w = np.array([2.5, -1.3, 0.7])
true_b = 4.0
noise = np.random.randn(200) * 0.5
y = X @ true_w + true_b + noise
```

4.1. Macierz cech

`np.random.randn(200, 3)` losuje 600 wartości z rozkładu normalnego standardowego:

$$X_{ij} \sim \mathcal{N}(0, 1).$$

Każda z trzech kolumn ma w populacji średnią 0 i wariancję 1. W skończonej próbie wartości będą tylko zbliżone do tych liczb.

Typowa obserwacja może wyglądać tak:

$$x_i = (0.8, -0.4, 1.2).$$

Bez szumu model wygenerowałby wtedy:

$$2.5(0.8) - 1.3(-0.4) + 0.7(1.2) + 4 = 2 + 0.52 + 0.84 + 4 = 7.36.$$

4.2. Prawdziwe współczynniki

```
true_w = np.array([2.5, -1.3, 0.7])
true_b = 4.0
```

Ich interpretacja przy założeniu *ceteris paribus* - pozostałe cechy są stałe - jest następująca:

- wzrost pierwszej cechy o 1 zwiększa oczekiwane y o 2.5,
- wzrost drugiej cechy o 1 zmniejsza oczekiwane y o 1.3,
- wzrost trzeciej cechy o 1 zwiększa oczekiwane y o 0.7,
- dla wszystkich cech równych zero oczekiwane y wynosi 4.

4.3. Szum

```
noise = np.random.randn(200) * 0.5
```

Jeżeli $z \sim \mathcal{N}(0, 1)$, to $0.5z \sim \mathcal{N}(0, 0.5^2)$. Zatem:

$$\varepsilon_i \sim \mathcal{N}(0, 0.25),$$

a równanie generujące dane jest równe:

$$y_i = x_i^\top w_{\text{true}} + b_{\text{true}} + \varepsilon_i.$$

Szum reprezentuje wszystko, czego nie opisują trzy cechy: błąd pomiaru, losowość i pominięte czynniki. Z jego powodu nawet idealnie działająca estymacja nie musi zwrócić dokładnie $[2.5, -1.3, 0.7]$ i 4.0 .

Ponieważ cechy są niezależne i mają wariancję 1, wariancja części sygnałowej wynosi w przybliżeniu:

$$\text{Var}(Xw) = \|w\|_2^2 = 2.5^2 + (-1.3)^2 + 0.7^2 = 8.43.$$

Wariancja szumu wynosi 0.25, więc sygnał jest w tym eksperymencie wyraźnie silniejszy od szumu. Te liczby są wartościami teoretycznymi; empiryczne wartości dla konkretnego losowania będą się różnić.

4.4. Brak pełnej powtarzalności

Kod nie wywołuje `np.random.seed(...)` ani nie korzysta z lokalnego generatora `np.random.default_rng(seed)`. Pole `seed=42` w klasie jest zapisywane, ale nieużywane. Każde uruchomienie może więc stworzyć inne dane i dać trochę inne wyniki. Ma to również wpływ na asercje.

5. Funkcja kosztu MSE

5.1. Od reszty do funkcji celu

Dla obserwacji i reszta wynosi:

$$e_i = \hat{y}_i - y_i.$$

Samą sumę błędów łatwo byłoby oszukać: błąd $+10$ i błąd -10 dałyby razem zero. Dlatego w **mean squared error** (MSE, średnim błędzie kwadratowym) błędy podnosimy do kwadratu:

$$\text{MSE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Kod jest bezpośrednim zapisem tego wzoru:

```
def mse(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    return np.mean((y_true - y_pred) ** 2)
```

Kolejność $y_true - y_pred$ nie wpływa na MSE, ponieważ:

$$(y_i - \hat{y}_i)^2 = (\hat{y}_i - y_i)^2.$$

Wpłynęłaby jednak na znak pochodnej przed podniesieniem do kwadratu, dlatego w wyprowadzeniach trzeba zachować spójną konwencję.

5.2. Postać macierzowa

Dla $\hat{y} = Xw + b\mathbf{1}$:

$$J(w, b) = \frac{1}{n} \|Xw + b\mathbf{1} - y\|_2^2.$$

Ponieważ $\|e\|_2^2 = e^\top e$, możemy też napisać:

$$J(w, b) = \frac{1}{n} (Xw + b\mathbf{1} - y)^\top (Xw + b\mathbf{1} - y).$$

Litera J oznacza **funkcję kosztu** albo **funkcję celu** (*cost function/objective function*). Jej argumentami są parametry modelu, a wynikiem jedna liczba. Trening ma znaleźć takie w, b , aby $J(w, b)$ było możliwie małe.

5.3. Dlaczego kwadrat?

Kwadrat ma kilka ważnych własności:

1. błędy dodatnie i ujemne nie znoszą się;
2. większe błędy są karane nieproporcjonalnie mocno;
3. funkcja jest gładka i różniczkowalna;
4. dla regresji liniowej otrzymujemy wypukły problem optymalizacyjny;
5. MSE wynika naturalnie z założenia o normalnym rozkładzie szumu.

Wadą jest duża wrażliwość na obserwacje odstające. Błąd 10 wnosi do sumy 100, podczas gdy błąd 1 wnosi tylko 1.

5.4. Interpretacja probabilistyczna

Założmy:

$$y_i = x_i^\top w + b + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

Wtedy:

$$p(y_i | x_i, w, b) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i^\top w - b)^2}{2\sigma^2}\right).$$

Przy niezależnych obserwacjach likelihood jest iloczynem tych gęstości. Po zastosowaniu logarytmu iloczyn zmienia się w sumę. Maksymalizacja log-likelihood jest - po odrzuceniu stałych niezależnych od w, b - równoważna minimalizacji:

$$\sum_{i=1}^n (y_i - x_i^\top w - b)^2.$$

Dlatego estymator najmniejszych kwadratów jest jednocześnie estymatorem największej wiarygod-

ności przy normalnym szumie.

5.5. MSE a czynnik 1/2

W materiałach można spotkać:

$$J = \frac{1}{2n} \sum_i e_i^2.$$

Czynnik 1/2 usuwa później dwójkę z gradientu. Nie zmienia położenia minimum, tylko skalę funkcji i gradientu. Projekt używa prawdziwego MSE z czynnikiem 1/n, dlatego w `d_out` pojawia się `2 / y_batch.size`.

6. Gradient MSE - pełne wyprowadzenie

6.1. Co oznacza gradient?

Gradient funkcji skalarnej wielu zmiennych jest wektorem pochodnych cząstkowych:

$$\nabla_w J = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \vdots \\ \frac{\partial J}{\partial w_d} \end{bmatrix}.$$

Wskazuje kierunek najszybszego lokalnego wzrostu funkcji. Wobec tego $-\nabla J$ jest kierunkiem najszybszego lokalnego spadku w sensie normy euklidesowej.

„Lokalny” jest ważnym słowem: gradient opisuje zachowanie funkcji w małym otoczeniu aktualnego punktu przez przybliżenie pierwszego rzędu:

$$J(\theta + \Delta\theta) \approx J(\theta) + \nabla J(\theta)^\top \Delta\theta.$$

6.2. Wyprowadzenie skalarne

Mamy:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (x_i^\top w + b - y_i)^2.$$

Oznaczmy:

$$e_i = x_i^\top w + b - y_i.$$

Dla współczynnika w_j , używając reguły łańcuchowej:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n 2e_i \frac{\partial e_i}{\partial w_j}.$$

Ponieważ:

$$e_i = \sum_{k=1}^d x_{ik} w_k + b - y_i,$$

to:

$$\frac{\partial e_i}{\partial w_j} = x_{ij}.$$

Ostatecznie:

$$\frac{\partial J}{\partial w_j} = \frac{2}{n} \sum_{i=1}^n x_{ij}(\hat{y}_i - y_i).$$

Dla biasu $\partial e_i / \partial b = 1$, więc:

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i).$$

6.3. Wyprowadzenie macierzowe

Niech:

$$e = Xw + b\mathbf{1} - y.$$

Wtedy:

$$J = \frac{1}{n} e^\top e.$$

Różniczka wynosi:

$$dJ = \frac{1}{n} ((de)^\top e + e^\top de).$$

Oba składniki są skalarami i są sobie równe, dlatego:

$$dJ = \frac{2}{n} e^\top de.$$

Przy różniczkowaniu względem w mamy $de = X dw$, zatem:

$$dJ = \frac{2}{n} e^\top X dw = \left(\frac{2}{n} X^\top e \right)^\top dw.$$

Z definicji gradientu $dJ = (\nabla_w J)^\top dw$, więc:

$$\nabla_w J = \frac{2}{n} X^\top (Xw + b\mathbf{1} - y).$$

Analogicznie dla b , ponieważ $de = \mathbf{1} db$:

$$\frac{\partial J}{\partial b} = \frac{2}{n} \mathbf{1}^\top (Xw + b\mathbf{1} - y).$$

6.4. Przełożenie na kod

```
y_pred = X_batch @ self.w + self.b
d_out = 2 / y_batch.size * (y_pred - y_batch)
grad_w = X_batch.T @ d_out
grad_b = d_out.sum()
```

`d_out` jest pochodną kosztu względem wyjścia modelu:

$$d_out = \frac{\partial J}{\partial \hat{y}} = \frac{2}{m} (\hat{y} - y),$$

gdzie m oznacza wielkość aktualnego batcha. Następnie:

$$\frac{\partial \hat{y}}{\partial w} = X,$$

więc reguła łańcuchowa daje:

$$\nabla_w J = X^\top \mathbf{d}_{\text{out}}.$$

Dla biasu każda predykcja ma pochodną równą 1, dlatego sumujemy elementy $\mathbf{d}_{\text{out}}$.

Kontrola wymiarów

$$X_{\text{batch}}^\top \in \mathbb{R}^{d \times m}, \quad \mathbf{d}_{\text{out}} \in \mathbb{R}^m,$$

zatem:

$$X_{\text{batch}}^\top \mathbf{d}_{\text{out}} \in \mathbb{R}^d,$$

czyli gradient ma dokładnie taki sam wymiar jak w . Jest to uniwersalna kontrola implementacji: gradient parametru musi mieć kształt zgodny z tym parametrem.

7. Geometria, wypukłość i Hessian

7.1. Powierzchnia funkcji kosztu

Dla pojedynczej wagi i biasu $J(w, b)$ jest powierzchnią nad płaszczyzną parametrów. Dla regresji liniowej z MSE ma kształt paraboloidy - odpowiednika paraboli w większej liczbie wymiarów.

Poziomice funkcji są zwykle elipsami. Jeżeli elipsy są bardzo wydłużone, gradient wskazuje głównie kierunek dużej krzywizny. Zwykły GD może wtedy zygzakować zamiast szybko poruszać się wzdłuż doliny ku minimum.

7.2. Hessian

Pomijając na chwilę bias:

$$J(w) = \frac{1}{n} \|Xw - y\|_2^2,$$

$$\nabla J(w) = \frac{2}{n} X^\top (Xw - y),$$

$$H = \nabla^2 J(w) = \frac{2}{n} X^\top X.$$

Macierz $X^\top X$ jest dodatnio półokreślona, ponieważ dla każdego z :

$$z^\top X^\top X z = \|Xz\|_2^2 \geq 0.$$

Z tego wynika wypukłość MSE względem parametrów regresji liniowej. Każde minimum lokalne jest minimum globalnym. Jeśli kolumny macierzy projektu mają pełną rangę, Hessian jest dodatnio określony i minimum jest jednoznaczne.

7.3. Wartości własne i learning rate

Wartości własne Hessianu opisują krzywiznę w jego głównych kierunkach. Dla kwadratowej funkcji celu stały learning rate daje zbieżność przy odpowiednio małym kroku. W uproszczonej sytuacji:

$$0 < \eta < \frac{2}{\lambda_{\max}(H)}.$$

Ponieważ $H = \frac{2}{n} X^\top X$, granica zależy od skali danych. Duże wartości cech powodują duże wartości własne Hessianu, a więc wymagają mniejszego learning rate.

To nie jest arbitralna zasada praktyczna. Jest to konsekwencja dynamiki aktualizacji. Dla kierunku własnego o wartości własnej λ odległość od optimum jest w kolejnym kroku mnożona przez:

$$1 - \eta\lambda.$$

- gdy wartość bezwzględna jest mniejsza od 1, odległość maleje;
- gdy jest ujemna, kolejne punkty przeskakują na przeciwne strony minimum;
- gdy jej wartość bezwzględna przekracza 1, iteracje rozbiegają się.

7.4. Uwarunkowanie i skalowanie cech

Condition number (wskaźnik uwarunkowania) dla dodatnio określonego Hessianu jest ilorazem:

$$\kappa(H) = \frac{\lambda_{\max}(H)}{\lambda_{\min}(H)}.$$

Duże κ oznacza bardzo różne krzywizny w różnych kierunkach. Jeden learning rate musi być mały ze względu na kierunek stromy, przez co postęp w kierunku płaskim jest powolny.

Standaryzacja cech nie zawsze usuwa wszystkie problemy z uwarunkowaniem, zwłaszcza przy silnej korelacji, ale często bardzo je zmniejsza. To dlatego dane California Housing są skalowane przed treningiem przez GD.

8. Rozwiązanie analityczne i np.linalg.lstsq

8.1. Włączenie biasu do macierzy

Kod tworzy kolumnę jedynek i dołącza ją do X :

```
ones = np.ones((X.shape[0], 1))
X_aug = np.hstack([X, ones])
```

Otrzymujemy **augmented design matrix** (rozszerzoną macierz projektu):

$$X_{\text{aug}} = \begin{bmatrix} X & \begin{matrix} 1 \\ 1 \\ \vdots \\ 1 \end{matrix} \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}.$$

Parametry łączymy w jeden wektor:

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix}.$$

Wtedy model można zapisać prościej:

$$\hat{y} = X_{\text{aug}}\theta.$$

8.2. Normal equations

Minimalizujemy:

$$J(\theta) = \frac{1}{n} \|X_{\text{aug}}\theta - y\|_2^2.$$

Gradient wynosi:

$$\nabla_{\theta} J = \frac{2}{n} X_{\text{aug}}^{\top} (X_{\text{aug}}\theta - y).$$

W minimum gradient jest równy zero:

$$X_{\text{aug}}^{\top} (X_{\text{aug}}\theta - y) = 0,$$

czyli:

$$X_{\text{aug}}^{\top} X_{\text{aug}} \theta = X_{\text{aug}}^{\top} y.$$

Są to **normal equations** (równania normalne). Jeżeli $X_{\text{aug}}^{\top} X_{\text{aug}}$ jest odwracalna:

$$\hat{\theta} = (X_{\text{aug}}^{\top} X_{\text{aug}})^{-1} X_{\text{aug}}^{\top} y.$$

8.3. Dlaczego kod nie oblicza odwrotności?

Kod używa:

```
coef = np.linalg.lstsq(X_aug, y, rcond=None)[0]
```

`lstsq` oznacza **least squares** - najmniejsze kwadraty. Funkcja rozwiązuje problem:

$$\min_{\theta} \|X_{\text{aug}} \theta - y\|_2.$$

Nie ma potrzeby jawnie tworzyć $(X^{\top} X)^{-1}$. Jawne równania normalne:

1. mogą zwiększać problemy numeryczne, ponieważ wskaźnik uwarunkowania $X^{\top} X$ jest w przybliżeniu kwadratem wskaźnika dla X ;
2. nie działają przy osobliwości macierzy;
3. wykonują niepotrzebne obliczenie odwrotności, gdy wystarcza rozwiązanie układu.

Algorytmy least squares mogą korzystać z rozkładów takich jak QR lub SVD. **SVD** (*singular value decomposition*, rozkład według wartości osobliwych) pozwala opisać także macierze bez pełnej rangi. Koncepcyjnie rozwiązanie można zapisać przez pseudoodwrotność Moore'a-Penrose'a:

$$\hat{\theta} = X_{\text{aug}}^+ y.$$

Jeśli rozwiązań minimalizujących reszty jest wiele, standardowe rozwiązanie least squares wybiera rozwiązanie o najmniejszej normie euklidesowej.

Funkcja zwraca kilka elementów: rozwiązanie, sumy kwadratów reszt, rangę macierzy i jej wartości osobliwe. Indeks `[0]` wybiera pierwszy element, czyli wektor współczynników. Argument `rcond=None` pozostawia bibliotece standardowy próg określający, kiedy bardzo małą wartość osobliwą traktować numerycznie jak zero przy ocenie rangi.

8.4. Rozpakowanie wyniku

```
self.w = coef[:-1]
self.b = coef[-1]
```

Ponieważ kolumna jedynek została dołączona jako ostatnia, ostatni element `coef` odpowiada biasowi. Wszystkie wcześniejsze elementy są wagami.

8.5. Closed form kontra gradient descent

Closed-form solution oznacza rozwiązanie otrzymane bez iteracyjnego wykonywania małych kroków optymalizacji. Nie oznacza, że komputer dosłownie wstawia liczby do prostego wzoru z odwrotnością - `lstsq` nadal wykonuje algorytm numeryczny.

Właściwość	Least squares	Gradient descent
Iteracje/epoki	Brak epok użytkownika	Wymaga wielu kroków
Learning rate	Niepotrzebny	Trzeba dobrać
Dokładność dla małego problemu	Zwykle bardzo wysoka	Zależy od liczby kroków
Duże d	Rozkład macierzy może być kosztowny	Każdy krok może być tańszy

Właściwość	Least squares	Gradient descent
Dane strumieniowe	Nie jest naturalnym wyborem	SGD może uczyć się na bieżąco
Wartość edukacyjna dla deep learningu	Ograniczona	Bardzo duża

W regresji liniowej oba podejścia powinny prowadzić do tego samego minimum, jeśli GD zbiegnie i oba minimalizują dokładnie tę samą funkcję celu.

9. Gradient descent

9.1. Podstawowa reguła

Gradient descent (spadek gradientowy) jest iteracyjnym algorytmem minimalizacji funkcji. Dla całego wektora parametrów θ :

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

gdzie:

- t - numer kroku,
- $\eta > 0$ - **learning rate** (współczynnik uczenia, długość kroku),
- $\nabla J(\theta_t)$ - gradient obliczony w aktualnym punkcie.

Gradient odejmujemy, ponieważ wskazuje wzrost funkcji. Dla małego η :

$$J(\theta - \eta \nabla J) \approx J(\theta) - \eta \|\nabla J\|_2^2,$$

więc koszt powinien się zmniejszyć.

9.2. Co może się wydarzyć podczas treningu?

Learning rate za mały

- loss maleje bardzo wolno,
- model po ustalonej liczbie epok pozostaje daleko od optimum,
- krzywa może wyglądać prawie płasko.

Learning rate odpowiedni

- loss szybko i stabilnie maleje,
- wagi zbliżają się do optimum,
- kolejne zmiany stają się coraz mniejsze.

Learning rate za duży

- parametry przeskakują nad minimum,
- loss oscyluje,
- wartości mogą rosnąć,
- może pojawić się `inf` lub `nan` wskutek przepełnienia numerycznego.

Gradient bliski zeru

W regresji liniowej z MSE oznacza zwykle bliskość minimum albo płaski kierunek wynikający z braku identyfikacji. W głębokich sieciach mały gradient może też wynikać z problemu zanikającego gradientu, ale ten projekt jeszcze tego zjawiska nie demonstruje.

9.3. Inicjalizacja zerami

Kod ustawia:

```
self.w = np.zeros(X.shape[1])
self.b = 0.0
```

Dla liniowego, wypukłego problemu jest to całkowicie poprawne. Nie występuje problem symetrii znany z wielowarstwowych sieci neuronowych. Jeżeli gradient w zerze nie jest zerowy, parametry rozpoczną ruch.

Początkowa predykcja wynosi zero dla każdej obserwacji. Pierwszy gradient wag jest równy:

$$\nabla_w J(0, 0) = -\frac{2}{n} X^\top y.$$

Zależy więc od korelacji cech z targetem.

9.4. Epoka nie jest tym samym co krok

Epoch (epoka) oznacza jedno przejście przez cały zbiór treningowy. **Optimization step** oznacza jedną aktualizację parametrów.

Jeśli jest 200 obserwacji:

- `batch_size=None`: 1 krok na epokę,
- `batch_size=32`: 7 kroków na epokę, bo $\lceil 200/32 \rceil = 7$,
- `batch_size=1`: 200 kroków na epokę.

Zatem dwa modele trenowane przez 100 epok mogą wykonać odpowiednio 100 i 700 aktualizacji. Porównanie ich wyłącznie względem numeru epoki nie oddziela wpływu batch size od liczby aktualizacji.

10. Batch GD, mini-batch GD i SGD

10.1. Full-batch gradient descent

Full-batch GD oblicza gradient na całym zbiorze:

$$g = \frac{2}{n} X^\top (Xw + b\mathbf{1} - y).$$

W projekcie występuje, gdy `batch_size` is `None`, ponieważ:

```
batch_size = self.batch_size if self.batch_size is not None else X.shape[0]
```

Gradient jest deterministyczny dla ustalonych danych i parametrów. Krzywa loss powinna być stosunkowo gładka.

10.2. Mini-batch gradient descent

Mini-batch to niewielki podzbiór obserwacji, np. 32 rekordy. Gradient:

$$g_B = \frac{2}{|B|} X_B^\top (X_B w + b\mathbf{1} - y_B)$$

jest przybliżeniem gradientu pełnego zbioru. Przy losowym próbkowaniu jest estymatorem nieobciążonym:

$$\mathbb{E}[g_B] = g.$$

Ma jednak wariancję, dlatego parametry wykonują bardziej zaszumioną trajektorię. Szum może być wadą, ale w złożonych problemach pomaga wykonywać wiele szybkich aktualizacji i bywa korzystny dla generalizacji.

10.3. Stochastic gradient descent

SGD (*stochastic gradient descent*, stochastyczny spadek gradientowy) w ścisłym znaczeniu wykorzystuje jedną obserwację na krok:

$$g_i = 2x_i(x_i^\top w + b - y_i).$$

W praktyce nazwa SGD bywa używana szerzej także wobec losowego mini-batch GD.

W tym projekcie `batch_size=1` dałoby klasyczny wariant SGD, mimo że nie ma osobnej wartości `method="sgd"`.

10.4. Slicing batchy

```
for batch in range(0, X.shape[0], batch_size):
    X_batch = X[batch : batch + batch_size]
    y_batch = y[batch : batch + batch_size]
```

Dla $n = 200$ i `batch_size=32` początki batchy to:

```
0, 32, 64, 96, 128, 160, 192
```

Ostatnie wycinki `X[192:224]` i `y[192:224]` nie powodują błędów. Python zatrzymuje się na końcu tablicy, więc ostatni batch ma 8 obserwacji.

10.5. Dlaczego tasowanie jest ważne?

Kod przechodzi przez dane zawsze w tej samej kolejności i ich nie tasuje. Dla niezależnie wygenerowanych danych syntetycznych kolejność jest przypadkowa już na wejściu. W realnym zbiorze rekordy mogą być uporządkowane według czasu, klasy lub wartości targetu.

Bez tasowania kolejne gradienty mogą być systematycznie skrzywione w różnych częściach epoki. Standardową praktyką jest utworzenie losowej permutacji indeksów przed każdą epoką. Wyjątkiem są niektóre dane czasowe, gdzie sposób próbkowania trzeba zaprojektować tak, aby nie naruszyć struktury czasu.

10.6. Subtelność `loss_history`

Kod robi:

```
batch_loss.append(mse(y_batch, y_pred))
...
self.loss_history.append(np.mean(batch_loss))
```

Są tu trzy subtelności:

1. `y_pred` powstało **przed** aktualizacją parametrów dla danego batcha.
2. Kolejne wartości w `batch_loss` odpowiadają różnym stanom modelu.
3. Zwykła średnia nadaje ostatniemu batchowi o rozmiarze 8 taką samą wagę jak batchom o rozmiarze 32.

Dlatego zapisana liczba nie jest dokładnie MSE całego zbioru dla parametrów z końca epoki. Dla celów demonstracyjnych może pokazywać ogólny trend, ale precyzyjniejszy monitoring wykonałby po epoce:

```
epoch_pred = self.predict(X)
epoch_loss = mse(y, epoch_pred)
```

Nie trzeba zmieniać projektu, aby go zrozumieć; trzeba tylko poprawnie interpretować wykres.

11. Implementacja klasy `LinearRegression`

11.1. Konstruktor

```
class LinearRegression:
    def __init__(
```

```

        self,
        method: str = "gd",
        lr: float = 0.01,
        n_epochs: int = 100,
        batch_size: int | None = None,
        alpha: float = 0.0,
        optimizer: str = "vanilla",
        seed: int = 42,
    ) -> None:

```

Konstruktor zapisuje konfigurację modelu. Adnotacje typów pomagają człowiekowi, IDE i narzędziom statycznym, lecz Python domyślnie nie wymusza ich podczas działania.

Argument	Znaczenie	Wartość domyślna
method	sposób dopasowania: iteracyjny GD albo least squares	"gd"
lr	learning rate	0.01
n_epochs	liczba przejść przez dane	100
batch_size	liczba obserwacji na aktualizację	None, czyli pełny zbiór
alpha	siła kary Ridge	0.0
optimizer	reguła aktualizacji parametrów	"vanilla"
seed	planowane ziarno losowości	42, ale obecnie nieużywane

Następnie:

```

self.w = None
self.b = None
self.loss_history = []
self.weight_history = []

```

`w` i `b` nie są znane przed `fit()`. Historie są pustymi listami, do których trening będzie dopisywał kolejne wartości.

W bardziej ścisłym typie hintingu można byłoby zaznaczyć `np.ndarray | None` i `float | None`. Obecna wersja jest dozwolona, ale mniej informacyjna.

11.2. fit

```

def fit(self, X: np.ndarray, y: np.ndarray) -> "LinearRegression":
    if self.method == "gd":
        self._fit_gd(X, y)
    elif self.method == "closed_form":
        self._fit_closed_form(X, y)
    return self

```

`fit` jest publicznym interfejsem treningu. Wybiera prywatną metodę na podstawie napisu. Zwrocenie `self` umożliwia **method chaining**, czyli:

```

model = LinearRegression(...).fit(X, y)

```

Zamiast tworzyć model w jednej linii i trenować go w następnej.

Adnotacja w cudzysłowie jest **forward reference** - odwołaniem do klasy, której definicja w tym miejscu jeszcze trwa. W nowszych stylach typowania można też użyć `Self`.

Jeśli `method` ma literówkę, kod nie zgłasza błędu: po prostu zwraca niedopasowany model. Jest to ważne miejsce do walidacji, ale nie wpływa na poprawne użycie pokazane w eksperymentach.

11.3. predict

```
def predict(self, X: np.ndarray) -> np.ndarray:
    return X @ self.w + self.b
```

Jest to dokładnie:

$$\hat{y} = Xw + b\mathbf{1}.$$

`X @ self.w` oblicza iloczyn skalarny każdego wiersza X z wektorem wag. Bias jest dodawany przez broadcasting.

Metoda zakłada, że:

- model został wcześniej dopasowany,
- liczba kolumn X odpowiada liczbie wag,
- dane są liczbowe i skończone.

Kod nie sprawdza tych warunków jawnie, więc niepoprawne użycie zakończy się komunikatem pochodzącym z NumPy albo Pythona.

11.4. score

```
def score(self, X: np.ndarray, y: np.ndarray) -> float:
    y_pred = self.predict(X)
    return r_squared(y, y_pred)
```

Metoda najpierw generuje predykcje, a następnie liczy współczynnik determinacji R^2 . Konwencja przypomina API scikit-learn, gdzie regresory również zwracają domyślnie R^2 w metodzie `score`.

11.5. Inicjalizacja stanu optymalizatorów

Na początku `_fit_gd`:

```
t = 0
self.b = 0.0
velocity_b = 0.0
v_b = 0.0
m_b = 0.0
self.w = np.zeros(X.shape[1])
velocity_w = np.zeros(X.shape[1])
v_w = np.zeros(X.shape[1])
m_w = np.zeros(X.shape[1])
```

Nie wszystkie zmienne są używane przez każdy optymalizator:

- Vanilla potrzebuje tylko parametrów i gradientu;
- Momentum potrzebuje `velocity_w` i `velocity_b`;
- Adam potrzebuje `m_w`, `v_w`, `m_b`, `v_b` oraz `t`.

Ich inicjalizacja naraz upraszcza kod. `t` w Adamie liczy kroki optymalizatora, nie epoki. Przy mini-batchach rośnie kilka razy na epokę.

11.6. Pętla treningowa

```
for _ in range(self.n_epochs):
```

Zmienna `_` sygnalizuje, że numer epoki nie jest używany w ciele pętli.

Po obliczeniu gradientów kod wybiera aktualizację:

```

if self.optimizer == "vanilla":
    ...
elif self.optimizer == "momentum":
    ...
elif self.optimizer == "adam":
    ...

```

Wagi i bias są optymalizowane tą samą regułą, lecz mają osobny stan. Ten sam licznik t jest przekazywany dla wag i biasu, co jest poprawne - oba należą do tego samego kroku treningowego.

11.7. Historia wag i `.copy()`

```

self.weight_history.append(self.w.copy())

```

`.copy()` tworzy niezależną kopię wartości. Gdyby kod przechowywał referencję do tablicy modyfikowanej później w miejscu, historia mogłaby zawierać wielokrotnie ten sam końcowy stan. Aktualne optymalizatory zwracają nowe tablice zamiast modyfikować `self.w` operacją in-place, więc problem nie zawsze ujawniłby się obecnie, lecz kopia jest bezpieczną i czytelną decyzją.

Historia zapisuje wagi raz na końcu każdej epoki. Nie zapisuje początkowego wektora zerowego ani biasu.

12. Vanilla GD, Momentum i Adam

12.1. Wspólna idea optymalizatora

Optimizer (optymalizator) jest regułą przekształcającą gradient i dotychczasowy stan w nowe parametry. Sam optimizer nie wie, czy optymalizuje regresję, sieć neuronową czy funkcję dwóch zmiennych. Otrzymuje liczby i wykonuje aktualizację.

To rozdzielenie widać w `optimizers.py`: funkcje nie importują modelu ani MSE.

12.2. Vanilla gradient descent

```

def vanilla_step(params, grad, lr):
    return params - grad * lr

```

„Vanilla” oznacza wersję podstawową, bez dodatkowych mechanizmów:

$$\theta_t = \theta_{t-1} - \eta g_t.$$

Jeśli $\theta = 3$, gradient wynosi 8, a $\eta = 0.02$, to:

$$\theta_{\text{new}} = 3 - 0.02 \cdot 8 = 2.84.$$

Duży gradient daje duży krok. To może przyspieszać ruch daleko od minimum, ale może również powodować niestabilność w kierunkach o dużej krzywiznie.

12.3. Momentum

Kod:

```

velocity = beta * velocity + (1 - beta) * grad
params = params - lr * velocity

```

Matematycznie:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t,$$

$$\theta_t = \theta_{t-1} - \eta v_t.$$

`velocity` oznacza prędkość. Jest to wykładnicza średnia ruchoma gradientów. Po rozwinięciu rekurencji przy $v_0 = 0$:

$$v_t = (1 - \beta) \sum_{k=1}^t \beta^{t-k} g_k.$$

Nowsze gradienty mają większą wagę, a starsze zanikają geometrycznie. Dla $\beta = 0.9$ gradient sprzed 10 kroków ma mnożnik proporcjonalny do $0.9^{10} \approx 0.349$.

Przybliżona efektywna długość pamięci średniej wykładniczej jest rzędu $1/(1-\beta)$. Dla $\beta = 0.9$ jest to około 10 kroków. Jest to intuicyjna skala, a nie ostra granica - starsze gradienty nadal mają niezerową, malejącą wagę.

Intuicja

- Jeśli gradienty konsekwentnie wskazują podobny kierunek, prędkość się utrzuja.
- Jeśli znak gradientu szybko oscyluje, dodatnie i ujemne wkłady częściowo się wygaszają.
- W wydłużonej dolinie Momentum może ograniczyć zygzakowanie i przyspieszyć ruch w kierunku minimum.

Dwie spotykane konwencje

Projekt używa średniej:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t.$$

Popularna jest również wersja:

$$v_t = \beta v_{t-1} + g_t.$$

Nie są numerycznie identyczne dla tego samego learning rate. W stanie ustalonym przy stałym gradiencie pierwsza wersja dąży do g , a druga do $g/(1-\beta)$. Obie mogą działać, lecz learning rate trzeba interpretować razem z wybraną konwencją.

Pierwszy krok

Przy $v_0 = 0$ i $\beta = 0.9$:

$$v_1 = 0.1g_1.$$

Pierwszy krok projektu jest zatem dziesięć razy mniejszy niż pierwszy krok Vanilla przy tym samym learning rate i gradiencie. W kolejnych krokach prędkość narasta.

12.4. Adam

Adam pochodzi od *adaptive moment estimation* - adaptacyjnego szacowania momentów. Łączy średnią gradientów podobną do Momentum z adaptacyjnym skalowaniem każdego parametru.

Kod pierwszego momentu:

```
m = beta1 * m + (1 - beta1) * grad
```

czyli:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t.$$

`m` jest wygładzoną średnią gradientów - estymatą pierwszego surowego momentu.

Kod drugiego momentu:

```
v = beta2 * v + (1 - beta2) * grad**2
```

czyli:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Kwadrat jest liczony element po elemencie. v śledzi typową skalę kwadratów gradientów dla każdego parametru.

Przy domyślnych wartościach $\beta_1 = 0.9$ pierwszy moment reaguje na skalę mniej więcej kilkunastu ostatnich kroków, natomiast $\beta_2 = 0.999$ daje drugiemu momentowi znacznie dłuższą pamięć, rzędu około tysiąca kroków.

Korekta obciążenia początkowego

Ponieważ $m_0 = v_0 = 0$, początkowe średnie są sztucznie przyciągane do zera. Dla stałej średniej gradientu μ :

$$\mathbb{E}[m_t] = (1 - \beta_1^t) \mu.$$

Dlatego kod dzieli przez brakujący czynnik:

```
m_hat = m / (1 - beta1**t)
v_hat = v / (1 - beta2**t)
```

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Jest to **bias correction** - korekta obciążenia estymatora, a nie bias modelu regresji.

Aktualizacja

```
params = params - lr * m_hat / (np.sqrt(v_hat) + eps)
```

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Dzielenie jest elementowe. Jeśli dany parametr zwykle ma duże gradienty, $\sqrt{\hat{v}_t}$ jest duże i normalizuje krok. Jeśli gradienty są małe, mianownik jest mniejszy. Każdy parametr otrzymuje więc własną efektywną skalę kroku.

Rola eps

`eps=1e-8` zapobiega dzieleniu przez zero i poprawia stabilność numeryczną. Jest bardzo małe względem typowych wartości $\sqrt{\hat{v}}$, więc zwykle nie dominuje aktualizacji. Gdy drugi moment jest ekstremalnie mały, `eps` może jednak wpływać na efektywny krok.

Pierwszy krok Adama

Dla $t = 1$, pomijając `eps`, korekta sprawia, że:

$$\hat{m}_1 = g_1, \quad \hat{v}_1 = g_1^2.$$

Zatem:

$$\frac{\hat{m}_1}{\sqrt{\hat{v}_1}} \approx \frac{g_1}{|g_1|} = \text{sign}(g_1).$$

Pierwszy krok każdego parametru ma w przybliżeniu długość `lr`, niezależnie od wielkości niezerowego gradientu. To wyjaśnia, dlaczego Adam z `lr=0.02` może początkowo poruszać się znacznie wolniej niż

Vanilla na prostej funkcji, gdzie gradient ma wartość np. 30.

12.5. Typy skalarne i tablicowe

Funkcje są opisane jako przyjmujące `np.ndarray`, ale są używane również dla skalarnego `self.b`. Działania arytmetyczne działają dzięki NumPy i Pythonowi, lecz adnotacje nie opisują w pełni rzeczywistego zastosowania. To problem typowania, a nie błąd matematyczny.

13. Ridge i współliniowość

13.1. Konstrukcja cechy prawie zduplikowanej

Kod:

```
X_collinear = X.copy()
X_collinear[:, 0] = X_collinear[:, 0] + np.random.randn(200) * 0.01
X_extended = np.hstack([X, X_collinear[:, [0]]])
```

tworzy czwartą cechę:

$$x_4 = x_1 + \delta, \quad \delta \sim \mathcal{N}(0, 0.01^2).$$

Pierwsza i czwarta kolumna są niemal identyczne. Jest to **multicollinearity** (współliniowość): jedną cechę można prawie odtworzyć jako kombinację pozostałych.

13.2. Dlaczego współczynniki stają się niestabilne?

Model wykorzystuje sumę:

$$w_1 x_1 + w_4 x_4 \approx (w_1 + w_4) x_1.$$

Wiele par (w_1, w_4) może więc dawać prawie identyczne predykcje. Przykładowo sumę 2.5 można uzyskać jako:

$$(w_1, w_4) = (2.5, 0), (1.25, 1.25), (20, -17.5).$$

Małe zmiany danych mogą silnie zmieniać indywidualne współczynniki, choć predykcje pozostają podobne. Geometrycznie funkcja kosztu ma bardzo płaski kierunek. $X^\top X$ jest źle uwarunkowana, a przy idealnym duplikacie staje się osobliwa.

13.3. Kara Ridge

Ridge regression (regresja grzbietowa) dodaje karę za duże wagi:

$$J_{\text{Ridge}}(w, b) = \frac{1}{n} \|Xw + b\mathbf{1} - y\|_2^2 + \alpha \|w\|_2^2.$$

Ponieważ:

$$\|w\|_2^2 = \sum_{j=1}^d w_j^2,$$

gradient kary wynosi:

$$\nabla_w (\alpha \|w\|_2^2) = 2\alpha w.$$

Dokładnie ten składnik dodaje kod:

```
grad_w = grad_w + 2 * self.alpha * self.w
```

Bias nie jest karany. Jest to standardowa decyzja: bias ustala ogólny poziom predykcji, a nie siłę zależności od cech. Przy scentralizowanych danych karanie interceptu byłoby szczególnie mało uzasadnione.

13.4. Wpływ α

- $\alpha = 0$ daje zwykle least squares.
- Małe dodatnie α lekko stabilizuje współczynniki.
- Duże α silnie przyciąga wagi do zera.
- Dla $\alpha \rightarrow \infty$ wagi dążą do zera, choć bias może pozostać niezerowy.

Ridge wprowadza **bias-variance trade-off**. Estymator staje się obciążony, ale jego wariancja może istotnie spaść. Na nowych danych mniejsza wariancja może przeważać nad dodatkowym obciążeniem.

13.5. Rozwiązanie analityczne Ridge

Bez biasu albo po odpowiednim wycentrowaniu danych:

$$\hat{w}_{\text{Ridge}} = (X^\top X + n\alpha I)^{-1} X^\top y$$

dla funkcji celu użytej w projekcie. Czynniki n wynika z tego, że część MSE jest dzielona przez n , a kara nie. W innych bibliotekach definicja skali `alpha` może być inna.

Dodanie $n\alpha I$ zwiększa wartości własne macierzy w kierunkach wag i poprawia uwarunkowanie.

Metoda `_fit_closed_form` w projekcie nie uwzględnia `alpha`; Ridge jest zaimplementowany wyłącznie w ścieżce GD.

13.6. Ridge nie jest Lasso

Test używa:

```
np.abs(model_ridge.w).sum()
```

co jest normą L_1 :

$$\|w\|_1 = \sum_j |w_j|.$$

Natomiast Ridge karze kwadrat normy L_2 :

$$\|w\|_2^2 = \sum_j w_j^2.$$

Ridge często zmniejsza również sumę wartości bezwzględnych, więc asercja może przejść, ale nie jest bezpośrednim sprawdzeniem wielkości rzeczywiście karanej. **Lasso** używa kary L_1 i może ustawiać część wag dokładnie na zero. Ridge zwykle zmniejsza wagi płynnie, bez dokładnego zerowania.

13.7. Interpretacja bayesowska

Jeżeli przyjmiemy gaussowski prior na wagi:

$$w \sim \mathcal{N}(0, \tau^2 I),$$

to maksymalizacja rozkładu posterior prowadzi do funkcji celu zawierającej karę proporcjonalną do $\|w\|_2^2$. Ridge można więc interpretować jako założenie, że przed zobaczeniem danych spodziewamy się raczej umiarkowanych wag blisko zera niż bardzo dużych współczynników.

14. Metryki MSE i R^2

14.1. MSE jako loss i metryka

MSE pełni w projekcie dwie role:

- jest funkcją, której gradient służy do uczenia,
- jest wartością zapisywaną do monitorowania treningu.

Nie każda metryka musi być funkcją kosztu. Można trenować na MSE, a raportować także MAE lub R^2 .

MSE ma jednostkę będącą kwadratem jednostki targetu. Jeśli y jest mierzone w złotych, MSE jest w złotych kwadratowych. Często raportuje się **RMSE**:

$$\text{RMSE} = \sqrt{\text{MSE}},$$

które ma tę samą jednostkę co y .

14.2. Współczynnik determinacji

Kod:

```
ss_res = np.sum((y_true - y_pred) ** 2)
ss_tot = np.sum((y_true - y_true.mean()) ** 2)
return 1 - ss_res / ss_tot
```

Definiuje:

$$SS_{\text{res}} = \sum_i (y_i - \hat{y}_i)^2,$$

$$SS_{\text{tot}} = \sum_i (y_i - \bar{y})^2,$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}.$$

SS_{res} to niewyjaśniona suma kwadratów reszt. SS_{tot} mierzy całkowite zróżnicowanie targetu wokół jego średniej.

R^2 porównuje model z prostym predyktorem, który dla każdej obserwacji zwraca średnią $\bar{y}$.

14.3. Interpretacja

- $R^2 = 1$: perfekcyjne predykcje, $SS_{\text{res}} = 0$.
- $R^2 = 0$: model ma taki sam błąd kwadratowy jak stała predykcja $\bar{y}$.
- $R^2 < 0$: model jest gorszy niż przewidywanie średniej.

R^2 nie jest ogólnie ograniczone od dołu przez zero. Może wynosić np. -3 . W klasycznym OLS z interceptem ocenianym na tym samym zbiorze treningowym R^2 nie jest ujemne, ponieważ rozwiązanie co najmniej dorównuje predykcji średniej. Na zbiorze testowym lub dla niedotrenowanego GD może być ujemne.

Stwierdzenie „ $R^2 = 0.6$ oznacza, że model wyjaśnia 60% wariancji” jest użytecznym skrótem w standardowym kontekście, ale nie powinno być automatycznie interpretowane przyczynowo.

14.4. Przypadek stałego targetu

Jeżeli każde y_i jest takie samo, to:

```
[ SS_{tot}=0.
]
```

Kod wykonuje wtedy dzielenie przez zero. Biblioteki muszą przyjąć konwencję albo zgłosić kontrolowany wyjątek. Projekt nie obsługuje tego szczególnego przypadku.

14.5. Ocena treningowa a generalizacja

Wysokie R^2 na danych treningowych mówi, jak dobrze model dopasował dane, które już widział. **Generalization** (generalizacja) oznacza jakość na nowych obserwacjach pochodzących z tego samego procesu.

Do jej oceny oddziela się dane treningowe i testowe. Wynik `model_housing.score(X_scaled, y)` jest wynikiem treningowym, ponieważ model był dopasowany do dokładnie tych samych `X_scaled` i `y`.

15. Eksperymenty z `run_experiments.py`

Funkcja `main()` pełni rolę demonstracji i prostego zestawu testów integracyjnych. Sprawdza nie tylko pojedyncze funkcje, ale cały przepływ: dane $\rightarrow$ trening $\rightarrow$ parametry $\rightarrow$ predykcje $\rightarrow$ wykres.

15.1. Importy

```
import numpy as np
from .model import LinearRegression
from sklearn.datasets import fetch_california_housing
from .optimizers import vanilla_step, momentum_step, adam_step
from .visualize import (...)
```

- NumPy odpowiada za obliczenia tablicowe i losowanie.
- `LinearRegression` jest własną implementacją modelu, nie klasą ze scikit-learn.
- Ze scikit-learn pobierany jest tylko zbiór danych.
- Optymalizatory są później testowane również niezależnie od regresji.
- Funkcje wykresów izolują kod prezentacji od obliczeń.

Kropka w `.model` oznacza import względny z tego samego pakietu. Z tego powodu plik zwykle uruchamia się jako moduł pakietu, a nie jako przypadkowy samodzielny skrypt.

15.2. Test rozwiązań analitycznego

```
model_cf = LinearRegression(method="closed_form").fit(X, y)
assert np.allclose(true_w, model_cf.w, atol=0.1)
assert np.isclose(true_b, model_cf.b, atol=0.1)
```

`np.allclose` porównuje wszystkie współrzędne wektorów, a `np.isclose` pojedyncze liczby. `atol=0.1` oznacza **absolute tolerance** - tolerancję bezwzględną 0.1, przy czym funkcje mają również domyślną tolerancję względną.

Asercje nie wymagają idealnej równości, ponieważ dane zawierają szum. Dla tego eksperymentu typowy błąd estymacji pojedynczej wagi jest rzędu:

$$\frac{\sigma}{\sqrt{n}} \approx \frac{0.5}{\sqrt{200}} \approx 0.035,$$

choć dokładna wariancja zależy od zrealizowanej macierzy X . Tolerancja 0.1 jest więc rozsądna, ale bez ustawionego ziarna losowego nie daje absolutnej gwarancji przejścia każdego losowania.

15.3. Test gradient descent

```
model_gd = LinearRegression(method="gd", n_epochs=1000).fit(X, y)
assert np.allclose(true_w, model_gd.w, atol=0.1)
assert np.isclose(true_b, model_gd.b, atol=0.1)
```

Domyślnie:

- `batch_size=None`, więc jest to full-batch GD,
- `optimizer="vanilla"`,

- `lr=0.01`,
- `alpha=0`.

Model wykonuje 1000 deterministycznych kroków na tym samym zbiorze. Jeśli learning rate zapewnia stabilność, powinien zbliżyć się do rozwiązania least squares.

Lepszy test zgodności obu metod porównałby również `model_gd.w` bezpośrednio z `model_cf.w`. Obecna asercja porównuje oba modele pośrednio z parametrami generującymi dane.

15.4. Full-batch kontra mini-batch

```
batch_gd = LinearRegression(method="gd").fit(X, y)
mini_batch_gd = LinearRegression(method="gd", batch_size=32).fit(X, y)
plot_loss_curves(batch_gd.loss_history, mini_batch_gd.loss_history)
```

Nazwę `batch_gd` należy rozumieć jako full-batch GD. Oba modele dostają 100 epok, ale:

- `batch_gd` wykonuje 100 aktualizacji,
- `mini_batch_gd` wykonuje 700 aktualizacji.

Mini-batch może więc dojść dalej nie tylko dzięki charakterowi gradientu, ale też dlatego, że dostał siedem razy więcej kroków. Jednocześnie pojedynczy krok używa mniejszej ilości danych.

Oczekiwany wykres:

- full-batch: bardziej gładki,
- mini-batch: potencjalnie szybszy spadek na epokę, ale większe wahania,
- obie krzywe: ogólny trend malejący przy odpowiednim `lr`.

Ze względu na sposób obliczania `loss_history` nie jest to idealnie kontrolowane porównanie naukowe. Jest jednak użyteczną demonstracją.

15.5. Eksperyment ze współliniowością i Ridge

```
model_ridge = LinearRegression(
    method="gd", alpha=0.1, n_epochs=1000
).fit(X_extended, y)

model_plain = LinearRegression(
    method="gd", alpha=0.0, n_epochs=1000
).fit(X_extended, y)
```

Oba modele widzą te same cztery cechy. Jedyna różnica to kara. Oczekujemy, że Ridge rozdzieli wpływ pomiędzy podobne cechy w sposób unikający dużych wag i zmniejszy normę wektora współczynników.

```
assert np.abs(model_ridge.w).sum() < np.abs(model_plain.w).sum()
```

Ten warunek często będzie prawdziwy, ale sprawdza normę L_1 , podczas gdy optymalizowana kara jest związana z kwadratem normy L_2 . Ponadto wynik zależy od skończonej liczby kroków GD. Asercja jest rozsądnym testem demonstracyjnym, nie twierdzeniem matematycznym obowiązującym dla dowolnych danych i ustawień.

15.6. Historia wag

```
plot_weight_convergence(np.array(model_gd.weight_history))
```

Lista stu lub tysiąca tablic `(d,)` zostaje zamieniona w jedną tablicę:

```
shape = (n_epochs, d).
```

Każda kolumna opisuje jedną wagę w czasie. Dla `model_gd` z 1000 epok i 3 cech kształt wynosi (1000, 3).

Krzywe powinny zaczynać się w pobliżu zera - pierwsza zapisana wartość jest już jednak stanem **po pierwszej epoce** - i zbliżać odpowiednio do 2.5, -1.3 i 0.7 z odchyleniami wynikającymi z szumu danych.

15.7. California Housing

```
data = fetch_california_housing()
X, y = data.data, data.target
```

Zbiór zawiera 20 640 obserwacji i 8 cech dotyczących obszarów w Kalifornii. Cechy obejmują m.in. medianę dochodów, wiek budynków, przeciętną liczbę pokoi, populację, przeciętne zaludnienie gospodarstwa oraz współrzędne geograficzne. Target opisuje medianę wartości domu w jednostkach 100 000 dolarów.

Jest to realny zbiór o znacznie mniej idealnej strukturze niż dane syntetyczne. Zależności nie muszą być liniowe, występują korelacje i złożone efekty przestrzenne.

Standaryzacja

```
X_scaled = (X - X.mean(axis=0)) / X.std(axis=0)
```

Dla każdej kolumny j :

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}.$$

`axis=0` oznacza agregację wzdłuż wierszy, oddzielnie dla każdej kolumny. `X.mean(axis=0)` i `X.std(axis=0)` mają kształt (8,). Broadcasting odejmuje odpowiednią średnią i dzieli przez odpowiednie odchylenie każdą kolumnę.

Po transformacji każda cecha ma na tym zbiorze średnią bliską 0 i odchylenie bliskie 1. Dzięki temu pojedynczy learning rate jest sensowny dla wszystkich wag.

Zmienia się też interpretacja współczynnika. Po standaryzacji waga w_j opisuje zmianę przewidywanego targetu przy wzroście cechy o jedno jej odchylenie standardowe, przy stałych pozostałych cechach.

Target nie jest standaryzowany. Nie jest to konieczne dla zwykłej regresji liniowej, choć jego skala wpływa na wielkości gradientów i początkowego loss.

Trening i wynik

```
model_housing = LinearRegression(method="gd", n_epochs=1000).fit(X_scaled, y)
r2 = model_housing.score(X_scaled, y)
y_pred = model_housing.predict(X_scaled)

assert 0.5 < r2 < 0.7
```

Zakres 0.5-0.7 jest zgodny z oczekiwaniem, że prosty model liniowy uchwyci dużą część sygnału, ale nie złożone nieliniowości. Nie należy jednak interpretować tej asercji jako jakości na niewidzianych danych - wynik jest liczony na zbiorze treningowym.

Gdyby cechy były dzielone na train i test, średnią i odchylenie należałoby obliczyć **wyłącznie na train**, a następnie użyć tych samych wartości do transformacji test. Obliczenie statystyk na całym zbiorze przed podziałem jest łagodną formą **data leakage** - przecieku informacji z testu.

15.8. Funkcja testowa dla optymalizatorów

```
def toy_function(params: np.ndarray) -> float:
    x, y = params[0], params[1]
```

```
return x**2 + 5 * y**2
```

Funkcja:

$$f(x, y) = x^2 + 5y^2$$

ma minimum globalne w $(0, 0)$ o wartości 0. Jej poziomice są elipsami. Kierunek y jest pięć razy mocniej ważony w funkcji niż kierunek x .

Gradient:

$$\nabla f(x, y) = \begin{bmatrix} 2x \\ 10y \end{bmatrix},$$

a Hessian:

$$H = \begin{bmatrix} 2 & 0 \\ 0 & 10 \end{bmatrix}.$$

Wskaźnik uwarunkowania wynosi $10/2 = 5$. Jest to problem anizotropowy - ma różną krzywiznę w różnych kierunkach - ale nie ekstremalnie trudny.

15.9. Zaszumiony gradient

```
def toy_gradient(params, noise_std=2.0):
    x, y = params[0], params[1]
    true_grad = np.array([2 * x, 10 * y])
    return true_grad + np.random.randn(2) * noise_std
```

Zamiast prawdziwego gradientu optimizer dostaje:

$$\tilde{g} = \nabla f(x, y) + \xi, \quad \xi \sim \mathcal{N}(0, 4I).$$

Jest to abstrakcyjna symulacja szumu gradientu znanego z mini-batchy. Daleko od minimum prawdziwy gradient, zwłaszcza jego składowa $10y$, jest duży względem szumu. Blisko zera prawdziwy gradient maleje, a szum o stałym odchyleniu 2 zaczyna dominować. Dlatego trajektoria nie musi osiąść dokładnie w minimum.

15.10. Vanilla na funkcji testowej

Bez szumu i przy $\eta = 0.02$:

$$x_{t+1} = x_t - 0.02(2x_t) = 0.96x_t,$$

$$y_{t+1} = y_t - 0.02(10y_t) = 0.8y_t.$$

Obie współrzędne maleją geometrycznie. Startując z $(3, 3)$:

$$(x_1, y_1) = (2.88, 2.4),$$

$$f(x_1, y_1) = 2.88^2 + 5(2.4)^2 = 37.0944,$$

wobec $f(3, 3) = 54$. Duży początkowy gradient y daje duży postęp w jednym kroku.

15.11. Momentum na funkcji testowej

Dla pierwszego kroku, bez szumu, $g_1 = (6, 30)$ i $\beta = 0.9$:

$$v_1 = 0.1g_1 = (0.6, 3),$$

$$(x_1, y_1) = (3, 3) - 0.02(0.6, 3) = (2.988, 2.94).$$

Pierwszy krok jest celowo wolniejszy z powodu konwencji EMA. W kolejnych krokach prędkość przechowuje historię gradientów.

15.12. Adam na funkcji testowej

W pierwszym kroku Adam po korekcie obciążenia normalizuje $g_1 = (6, 30)$ w przybliżeniu do $(1, 1)$. Zatem:

$$(x_1, y_1) \approx (2.98, 2.98).$$

Funkcja spada z 54 tylko do około:

$$6(2.98)^2 \approx 53.28.$$

Nie oznacza to, że Adam jest „gorszym optymalizatorem”. Oznacza, że przy wspólnym $lr=0.02$ jego znormalizowany krok jest mały względem bardzo korzystnego kroku Vanilla na tej prostej funkcji kwadratowej.

Komentarz w kodzie trafnie zauważa wpływ normalizacji, ale zdanie o przewadze Adama na „noisier or messier loss surfaces” jest uogólnieniem. Wynik pojedynczego eksperymentu zależy od:

- osobnego dostrojenia learning rate,
- skali i uwarunkowania funkcji,
- rozkładu szumu,
- liczby kroków,
- kryterium jakości końcowej.

Adam często jest wygodny w deep learningu, lecz nie ma uniwersalnego twierdzenia, że zawsze wygrywa na zaszumionych powierzchniach.

15.13. Czy porównanie jest uczciwe?

Każda z trzech pętli wywołuje `toy_gradient` osobno, więc dostaje inną sekwencję szumów. Różnice na wykresie wynikają z połączenia optymalizatora i losowania.

W kontrolowanym eksperymencie można:

- ustawić generator i resetować go do tego samego stanu przed każdym przebiegiem,
- przeprowadzić wiele losowych powtórzeń i porównać średnią z przedziałami niepewności,
- osobno dostroić learning rate każdego optymalizatora,
- raportować wynik względem liczby kroków i kosztu obliczeniowego.

Jest jeszcze subtelność: nawet przy tej samej sekwencji wektorów szumu optymalizatory odwiedzają inne punkty, więc ich prawdziwe gradienty są inne. Wspólny szum na numer kroku poprawia kontrolę eksperymentu, ale nie czyni trajektorii identycznymi poza działaniem optymalizatora.

15.14. Punkt wejścia programu

```
if __name__ == "__main__":  
    main()
```

Blok uruchamiania `main()` tylko wtedy, gdy moduł jest wykonywany jako punkt wejścia. Importowanie modułu nie uruchamia automatycznie eksperymentów. To ważne, ponieważ import powinien udostępniać definicje bez generowania danych i wykresów jako efektu ubocznego.

16. Wizualizacje

16.1. Katalog wynikowy

```
FIGURES_DIR = Path(__file__).parent / "figures"
```

`__file__` wskazuje ścieżkę bieżącego pliku. `.parent` wybiera jego katalog, a operator `/` obiektu `Path` buduje ścieżkę do podkatalogu `figures` w sposób niezależny od separatora systemu.

Każda funkcja wykonuje:

```
FIGURES_DIR.mkdir(exist_ok=True)
```

Katalog zostaje utworzony, jeśli nie istnieje. `exist_ok=True` zapobiega błędowi, jeśli już istnieje. Ponieważ tworzony jest bezpośredni podkatalog istniejącego katalogu modułu, `parents=True` nie jest tutaj konieczne.

16.2. fig i ax

```
fig, ax = plt.subplots()
```

- `fig` jest całym płótnem (*figure*), które można zapisać do pliku;
- `ax` jest obszarem osi (*axes*), na którym umieszcza się dane, etykiety i tytuł.

Rozróżnienie jest ważne: `ax.plot(...)` rysuje, a `fig.savefig(...)` zapisuje całą figurę.

16.3. Krzywe loss

```
ax.plot(batch_losses, label="batch GD")
ax.plot(minibatch_losses, label="mini-batch GD")
```

Jeśli nie podano osi x , Matplotlib używa indeksów 0, 1, 2, Tutaj indeks odpowiada epoce zapisanej w historii. `ax.legend()` pokazuje legendę na podstawie `label`.

Wykres służy do diagnozy:

- trend malejący sugeruje uczenie,
- oscylacje mogą wynikać z mini-batchy lub zbyt dużego learning rate,
- wzrost do ogromnych wartości sugeruje rozbieżność,
- plateau może oznaczać zbieżność albo zbyt mały krok.

Trzeba pamiętać o opisaniej wcześniej definicji `loss_history`.

16.4. Predykcje względem prawdy

```
ax.scatter(y_true, y_pred)
```

Każdy punkt ma współrzędne $(y_i, \hat{y}_i)$. Idealne predykcje leżą na prostej:

$$\hat{y} = y.$$

Kod wyznacza wspólny zakres:

```
lo = min(y_true.min(), y_pred.min())
hi = max(y_true.max(), y_pred.max())
ax.axline([lo, lo], [hi, hi])
```

Linia przechodzi przez (l_o, l_o) i (h_i, h_i) , a więc jest linią idealnej zgodności. Punkty nad nią oznaczają zawyżenie predykcji, a pod nią - zaniżenie.

Bez wymuszenia jednakowej skali osi linia może wizualnie nie tworzyć kąta 45° , choć matematycznie nadal ma nachylenie 1. Przy dużej liczbie punktów pomocne byłyby **alpha**, mniejszy rozmiar markerów i ewentualnie wykres reszt.

16.5. Zbieżność wag

```
for i in range(weight_history.shape[1]):
    ax.plot(weight_history[:, i], label=f"w{i}")
```

`weight_history[:, i]` wybiera wszystkie epoki dla jednej wagi. Etykiety zaczynają się od w_0 , zgodnie z indeksowaniem Pythona, chociaż w matematyce często piszemy w_1 .

Wykres pokazuje drogę parametrów, a nie bezpośrednio jakość predykcji. Wagi mogą nadal się zmieniać przy prawie stałym loss, zwłaszcza w źle uwarunkowanym lub współliniowym problemie.

16.6. Porównanie optymalizatorów

```
for label, loss_history in loss_histories.items():
    ax.plot(loss_history, label=label)
```

Słownik mapuje nazwę optymalizatora na listę wartości funkcji. Dzięki temu funkcja nie jest na stałe ograniczona do trzech serii.

Oś x oznacza tutaj krok, a nie epokę. Każda wartość jest obliczana **po** aktualizacji parametrów, w przeciwieństwie do `batch_loss` w modelu, które jest liczone przed aktualizacją danego batcha.

16.7. Zapisywanie i zarządzanie figurami

```
fig.savefig(FIGURES_DIR / "loss_curves.png")
```

Plik jest zapisany, lecz figura nie jest jawnie zamykana. Przy czterech wykresach zwykle nie stanowi to problemu. W większej pętli warto po zapisie wykonać:

```
plt.close(fig)
```

aby zwolnić zasoby. `fig.tight_layout()` może ograniczyć obcinanie etykiet, a argument `dpi` kontroluje rozdzielczość rastra.

17. Python, NumPy i struktura pakietu

17.1. Pusty `__init__.py`

Pusty plik nie wykonuje obliczeń, ale sygnalizuje, że katalog jest pakietem. Pozwala organizować kod i stosować importy względne:

```
from .metrics import r_squared, mse
```

Kropka oznacza „z bieżącego pakietu”. W typowej strukturze:

```
project/
  package_name/
    __init__.py
    model.py
```

```
metrics.py
optimizers.py
visualize.py
run_experiments.py
```

moduł można uruchomić z katalogu nadrzędnego poleceniem w rodzaju:

```
python -m package_name.run_experiments
```

Bezpośrednie `python package_name/run_experiments.py` może spowodować błąd importu względniego, ponieważ Python nie zawsze zna wtedy kontekst pakietu.

17.2. Adnotacje typów

Przykład:

```
batch_size: int | None = None
```

oznacza, że oczekiwana jest liczba całkowita albo `None`. -> `float` opisuje typ zwracany. Adnotacje:

- poprawiają dokumentację,
- wspierają autouzupełnianie,
- umożliwiają statyczną analizę,
- nie stanowią domyślnej walidacji w runtime.

`list[float]` i `dict[str, list[float]]` to generyczne typy kolekcji.

17.3. Operacje wektorowe zamiast pętli

```
y_pred = X_batch @ self.w + self.b
```

zastępuje pętle po obserwacjach i cechach. **Vectorization** (wektoryzacja) oznacza wyrażenie obliczeń jako operacji na całych tablicach. Kod jest krótszy, bliższy matematyce i zwykle szybszy, ponieważ ciężka praca odbywa się w zoptymalizowanych procedurach numerycznych.

17.4. Operacje elementowe a macierzowe

- `a * b` - mnożenie element po elemencie,
- `a ** 2` - potęgowanie każdego elementu,
- `A @ x` - iloczyn macierz-wektor,
- `X.T` - widok transponowany,
- `np.sum(...)` - agregacja,
- `np.mean(...)` - średnia.

W optymalizatorach `grad**2` musi być operacją elementową. Nie chodzi o iloczyn skalarny gradientu z samym sobą, ponieważ Adam potrzebuje osobnej estymaty skali dla każdego parametru.

17.5. `np.hstack` i zachowanie wymiaru

```
X_collinear[:, [0]]
```

ma shape `(200, 1)`, ponieważ indeks kolumny jest listą. Natomiast:

```
X_collinear[:, 0]
```

ma shape `(200,)`. Do poziomego dołączenia jako nowej kolumny potrzebny jest wariant dwuwymiarowy `(200, 1)`.

Podobnie `np.ones((X.shape[0], 1))` tworzy macierz kolumnową, a nie jednowymiarową tablicę.

17.6. Przypisanie wielu wartości

```
self.w, velocity_w = momentum_step(...)
```

Funkcja zwraca krotkę (`params`, `velocity`), a Python rozpakowuje ją do dwóch zmiennych. Trzeba zachować zarówno nowe parametry, jak i nowy stan optymalizatora. Gdyby `velocity_w` nie zostało zapisane, każdy krok zaczynałby z poprzednią, nieaktualną prędkością i Momentum nie działałoby zgodnie z definicją.

17.7. Stan modelu i stan optymalizatora

- `self.w`, `self.b` pozostają w obiekcie po zakończeniu `fit`, bo są potrzebne do predykcji.
- `velocity_*`, `m_*`, `v_*` są lokalne w `_fit_gd`, ponieważ po treningu projekt ich nie używa.
- `loss_history` i `weight_history` pozostają jako diagnostyka.

Ponowne wywołanie `fit()` inicjalizuje wagi i stan optimizerów od zera, ale nie czyści list historii. W efekcie nowe wpisy są dopisywane do starego treningu.

18. Założenia regresji i interpretacja statystyczna

Kod potrafi minimalizować MSE niezależnie od tego, czy klasyczne założenia statystyczne są spełnione. Założenia decydują jednak o interpretacji estymatora, jego własnościach i poprawności wnioskowania.

18.1. Liniowość w parametrach

Zakładamy:

$$y = Xw + \varepsilon$$

ewentualnie po dodaniu interceptu. Jeśli prawdziwa zależność jest silnie nieliniowa, model liniowy może mieć systematyczne reszty i wysoki **bias modelu**.

18.2. Egzogeniczność

Kluczowe założenie:

$$\mathbb{E}[\varepsilon \mid X] = 0.$$

Oznacza, że po uwzględnieniu cech błąd nie ma systematycznej części zależnej od X . Naruszenie może wynikać m.in. z pominiętej zmiennej skorelowanej z cechą, błędem pomiaru czy równoczesności.

Dla predykcji naruszenie klasycznych założeń inferencyjnych nie zawsze dyskwalifikuje model. Dla interpretacji przyczynowej współczynników egzogeniczność jest fundamentalna. Sam wysoki R^2 nie dowodzi przyczynowości.

18.3. Brak doskonałej współliniowości

Żadna kolumna macierzy projektu nie powinna być dokładną kombinacją liniową pozostałych, jeśli oczekujemy jednoznacznych współczynników OLS. Prawie doskonała współliniowość nie uniemożliwia obliczeń, lecz zwiększa wariancję współczynników.

18.4. Homoskedastyczność

Homoskedasticity oznacza stałą wariancję błędu warunkowo względem cech:

$$\text{Var}(\varepsilon_i \mid X) = \sigma^2.$$

Przy heteroskedastyczności OLS może nadal być nieobciążony pod odpowiednimi założeniami, ale klasyczne wzory błędów standardowych są niepoprawne. Kod nie oblicza błędów standardowych ani testów, więc nie dotyka bezpośrednio tego problemu.

18.5. Niezależność lub odpowiednia struktura zależności

W danych przekrojowych często zakłada się niezależne obserwacje. W szeregach czasowych błędy mogą być autokorelacyjne. Ponownie wpływa to szczególnie na wnioskowanie i sposób dzielenia danych.

18.6. Normalność błędów

Normalność nie jest konieczna do samego wyznaczenia OLS. Ułatwia dokładne wnioskowanie w małych próbach i daje interpretację MLE. Przy dużych próbach wiele wyników opiera się na asymptotyce.

18.7. Gauss-Markov

Przy standardowych założeniach klasyczny estymator OLS jest **BLUE** - *best linear unbiased estimator*, najlepszym liniowym estymatorem nieobciążonym. „Best” oznacza tu najmniejszą wariancję w klasie liniowych estymatorów nieobciążonych, a nie najlepszy możliwy model predykcyjny spośród wszystkich algorytmów.

18.8. Średnia i wariancja estymatora OLS

W modelu bez biasu w notacji albo z kolumną jedynek już zawartą w X :

$$y = Xw + \varepsilon.$$

Estymator OLS ma postać:

$$\hat{w} = (X^\top X)^{-1} X^\top y.$$

Po podstawieniu modelu generującego dane:

$$\hat{w} = (X^\top X)^{-1} X^\top (Xw + \varepsilon) = w + (X^\top X)^{-1} X^\top \varepsilon.$$

Jeżeli $\mathbb{E}[\varepsilon | X] = 0$, to:

$$\mathbb{E}[\hat{w} | X] = w,$$

czyli estymator jest warunkowo nieobciążony.

Przy homoskedastyczności i nieskorelowanych błędach:

$$\text{Var}(\varepsilon | X) = \sigma^2 I,$$

otrzymujemy:

$$\boxed{\text{Var}(\hat{w} | X) = \sigma^2 (X^\top X)^{-1}}.$$

Wzór pokazuje formalnie problem współliniowości: gdy $X^\top X$ ma bardzo małą wartość własną, jej odwrotność ma w tym kierunku bardzo dużą wartość własną, a wariancja współczynników rośnie.

Nieznana wariancję błędu często szacuje się przez:

$$\hat{\sigma}^2 = \frac{\text{SSE}}{n - p},$$

gdzie p jest liczbą estymowanych parametrów, łącznie z interceptem. Pierwiastki elementów diagonalnych macierzy

$$\hat{\sigma}^2 (X^\top X)^{-1}$$

są klasycznymi błędami standardowymi współczynników. Na ich podstawie konstruuje się statystyki t , przedziały ufności i testy hipotez. Projekt tego nie implementuje - skupia się na predykcji i optymalizacji - ale jest to ważne połączenie z ekonometrią.

18.9. OLS jako projekcja

W modelu bez problemów z rangą predykcja OLS:

$$\hat{y} = X\hat{w}$$

jest rzutem ortogonalnym y na przestrzeń kolumn macierzy X . Reszty:

$$e = y - \hat{y}$$

są ortogonalne do każdej kolumny X :

$$X^T e = 0.$$

Jest to dokładnie warunek zerowego gradientu i równania normalne. Nazwa „normalne” odnosi się tutaj do prostopadłości, nie do rozkładu normalnego.

Jeżeli macierz zawiera kolumnę jedynek, reszty są również ortogonalne do niej, więc:

$$\sum_i e_i = 0.$$

Oznacza to, że średnia reszt na zbiorze treningowym wynosi zero dla dokładnego rozwiązania OLS z interceptem.

19. Audyt projektu i miejsca do poprawy

Ten rozdział nie jest listą powodów, dla których projekt jest nieudany. Jak na pierwszy projekt ML, kod poprawnie implementuje centralne idee: predykcję liniową, MSE, gradient, trzy optymalizatory, rozwiązanie least squares i karę Ridge. Poniższe punkty pokazują różnicę między działającą demonstracją a odporną biblioteką lub kontrolowanym eksperymentem.

19.1. Co jest zrobione dobrze?

1. Matematyka podstawowego gradientu MSE jest poprawna.
2. Gradient Ridge ma poprawną postać dla zdefiniowanej funkcji celu.
3. Bias nie jest regularyzowany, zgodnie z typową konwencją.
4. Licznik Adama rośnie raz na batch i ten sam krok jest używany dla wag i biasu.
5. Adam zawiera korektę obciążenia obu momentów.
6. `np.linalg.lstsq` jest lepszym wyborem niż jawna odwrotność.
7. Kod jest logicznie rozdzielony na model, optymalizatory, metryki i wizualizację.
8. Wektoryzacja NumPy odpowiada bezpośrednio wzorom macierzowym.
9. Eksperymenty obejmują dane syntetyczne, realny dataset, współliniowość i niezależną funkcję testową.
10. Asercje zapewniają podstawowy sygnał, że implementacja robi to, czego oczekiwano.

19.2. Rzeczy wpływające na interpretację wyników

Historia loss

Średnia strat batchy nie jest dokładnym MSE modelu po epoce, ostatni batch ma nieproporcjonalną wagę, a kara Ridge nie jest dodawana do raportowanego loss. Gradient minimalizuje więc regularizowaną funkcję, podczas gdy wykres pokazuje tylko część predykcijną MSE.

To nie psuje aktualizacji parametrów, ale etykieta „loss” nie odpowiada całej funkcji celu przy `alpha > 0`.

Brak tasowania

Może zmieniać zachowanie mini-batch GD na uporządkowanych danych. Dla wylosowanych danych syntetycznych problem jest mniejszy.

Nierówna liczba kroków

Full-batch i mini-batch porównywane po tej samej liczbie epok wykonują inną liczbę aktualizacji. Wykres odpowiada na pytanie „co dzieje się po kolejnych przejściach przez dane?”, ale nie na pytanie „który algorytm jest lepszy po tej samej liczbie kroków?”.

Ocena na train

California Housing pokazuje dopasowanie, a nie jakość generalizacji.

Różne szumy optymalizatorów

Wynik porównania optimizerów ma dodatkową losową zmienność. Pojedyncza krzywa nie wystarcza do ogólnego rankingu.

19.3. Brak walidacji wejścia

Kod nie sprawdza jawnie:

- czy `X` jest dwuwymiarowe,
- czy `y` jest jednowymiarowe,
- czy liczby obserwacji są równe,
- czy dane nie są puste,
- czy nie zawierają `nan` lub `inf`,
- czy `batch_size > 0`,
- czy `method` i `optimizer` należą do obsługiwanych wartości,
- czy `lr > 0`, `n_epochs > 0` oraz `alpha >= 0`.

Przy poprawnym użyciu nie jest to przeszkoda. Przy błędnym użyciu komunikaty mogą być niejasne lub model może niczego nie aktualizować.

19.4. Nieznana metoda lub optimizer

Dla nieznanego `method` funkcja `fit` zwraca `self` bez treningu. Dla nieznanego `optimizer` gradient jest liczony, ale nie ma aktualizacji, a historia nadal jest zapisywana. Czytelny `ValueError` byłby bezpieczniejszy.

19.5. Predykcja przed treningiem

`self.w` i `self.b` są wtedy `None`, więc NumPy zgłosi błąd techniczny. Biblioteka zwykle zgłaszałaby specjalny błąd „model is not fitted”.

19.6. seed nie steruje losowością

Argument sugeruje powtarzalność, ale nie jest użyty ani przez klasę, ani przez generowanie danych. Jest więc obecnie martwym elementem interfejsu.

19.7. Ponowne fit

Wagi są resetowane, ale historie nie. Jeśli ten sam obiekt zostanie dopasowany ponownie, wykres połączy dwa niezależne treningi. Jest to zachowanie zaskakujące.

19.8. alpha i metoda closed form

Użytkownik może utworzyć:

```
LinearRegression(method="closed_form", alpha=0.1)
```

ale `alpha` zostanie zignorowane. Interfejs sugeruje możliwość, której metoda nie implementuje. W dokumencie trzeba zatem rozumieć `alpha` jako opcję ścieżki GD.

19.9. Test Ridge przez normę L1

Zmniejszenie $\|w\|_1$ jest prawdopodobnym skutkiem w tym przykładzie, ale logicznie bliższy test użyłby $\|w\|_2$ albo porównałby stabilność współczynników w wielu losowaniach.

19.10. Adnotacje typów optimizerów

Bias jest skalarem, mimo że sygnatury deklarują `np.ndarray`. Kod działa dynamicznie, ale statyczny checker może słusznie zgłosić niespójność.

19.11. Szczególny przypadek R^2

Stały target daje `ss_tot == 0`. Wartość R^2 nie ma wtedy standardowej interpretacji, a obecny kod może zwrócić ostrzeżenie i `nan` lub nieskończoność.

19.12. Zarządzanie wykresami

Brak `plt.close(fig)` jest nieistotny przy kilku wykresach, ale może zużywać pamięć przy setkach wywołań. Brak `tight_layout` może prowadzić do obcinania elementów przy innych tytułach lub ustawieniach.

19.13. Stopień ważności uwag

Kategoria	Przykłady	Czy blokuje obecny projekt?
Błąd interfejsu/odporności	brak błędu dla złej metody, predykcja przed <code>fit</code>	Nie przy poprawnym użyciu
Ograniczenie eksperymentu	train score, różne szumy, brak strojenia optimizerów	Nie, ale ogranicza wnioski
Subtelność metryki	sposób liczenia historii loss	Nie, ale zmienia interpretację wykresu
Brak funkcji	Ridge closed form, walidacja	Nie dla pokazanych eksperymentów
Drobna jakość kodu	typ biasu, zamykanie figur	Nie

Najważniejszy wniosek: rdzeń matematyczny projektu działa. Większość uwag dotyczy rzetelności eksperymentów, odporności API i precyzji diagnostyki.

20. Ręczny przykład liczbowy

Ten przykład łączy zapis skalarny, macierzowy i kod. Weźmy dwie obserwacje oraz dwie cechy:

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

Założmy parametry początkowe takie jak w projekcie:

$$w^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad b^{(0)} = 0.$$

20.1. Predykcja

$$\hat{y}^{(0)} = Xw^{(0)} + b^{(0)}\mathbf{1} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

W kodzie:

```
y_pred = X_batch @ self.w + self.b
```

20.2. Reszty i MSE

Przy konwencji używanej przez gradient:

$$e = \hat{y} - y = \begin{bmatrix} -5 \\ -6 \end{bmatrix}.$$

MSE:

$$J = \frac{1}{2} ((-5)^2 + (-6)^2) = \frac{61}{2} = 30.5.$$

20.3. d_out

Batch ma rozmiar $m = 2$, więc:

$$d_{\text{out}} = \frac{2}{m}(\hat{y} - y) = \frac{2}{2} \begin{bmatrix} -5 \\ -6 \end{bmatrix} = \begin{bmatrix} -5 \\ -6 \end{bmatrix}.$$

20.4. Gradient wag

$$X^T = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}.$$

Zatem:

$$\nabla_w J = X^T d_{\text{out}} = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} -5 \\ -6 \end{bmatrix} = \begin{bmatrix} -23 \\ -16 \end{bmatrix}.$$

Interpretacja znaku: zwiększenie obu wag powinno początkowo zmniejszać koszt, ponieważ gradient jest ujemny, a GD odejmuje gradient.

20.5. Gradient biasu

$$\frac{\partial J}{\partial b} = \sum_i (d_{\text{out}})_i = -5 - 6 = -11.$$

20.6. Jeden krok Vanilla GD

Dla $\eta = 0.01$:

$$w^{(1)} = w^{(0)} - 0.01 \nabla_w J = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.01 \begin{bmatrix} -23 \\ -16 \end{bmatrix} = \begin{bmatrix} 0.23 \\ 0.16 \end{bmatrix},$$

$$b^{(1)} = 0 - 0.01(-11) = 0.11.$$

Nowe predykcje:

$$\hat{y}^{(1)} = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} 0.23 \\ 0.16 \end{bmatrix} + 0.11 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.66 \\ 0.96 \end{bmatrix}.$$

Nowy MSE:

$$J^{(1)} = \frac{1}{2} ((0.66 - 5)^2 + (0.96 - 6)^2) \approx 22.12.$$

Koszt spadł z 30.5 do około 22.12.

20.7. Wpływ Ridge w tym samym punkcie

W pierwszym kroku $w^{(0)} = 0$, więc:

$$2\alpha w^{(0)} = 0.$$

Pierwszy krok Ridge jest identyczny jak bez regularyzacji. Dopiero gdy wagi odejdą od zera, kara zaczyna ciągnąć je z powrotem.

Na przykład dla $w = (0.23, 0.16)$ i $\alpha = 0.1$:

$$2\alpha w = 0.2 \begin{bmatrix} 0.23 \\ 0.16 \end{bmatrix} = \begin{bmatrix} 0.046 \\ 0.032 \end{bmatrix}.$$

Ten wektor zostaje dodany do gradientu danych.

20.8. Pierwszy krok Momentum

Dla gradientu $g = (-23, -16)$, $v_0 = 0$, $\beta = 0.9$:

$$v_1 = 0.9(0) + 0.1g = (-2.3, -1.6).$$

Przy $\eta = 0.01$:

$$w^{(1)} = 0 - 0.01v_1 = (0.023, 0.016).$$

Jest to dziesięć razy mniejszy krok niż Vanilla, zgodnie z konwencją zastosowaną w projekcie.

Założmy dla samej ilustracji, że w kolejnym kroku gradient pozostałby identyczny. Wtedy:

$$v_2 = 0.9(-2.3, -1.6) + 0.1(-23, -16) = (-4.37, -3.04).$$

Prędkość rośnie, ponieważ kolejne gradienty są zgodne.

20.9. Pierwszy krok Adama

Dla $g = (-23, -16)$, $m_0 = v_0 = 0$, $t = 1$:

$$m_1 = 0.1g = (-2.3, -1.6),$$

$$v_1 = 0.001g^2 = (0.529, 0.256).$$

Po korekcie:

$$\hat{m}_1 = \frac{m_1}{0.1} = (-23, -16),$$

$$\hat{v}_1 = \frac{v_1}{0.001} = (529, 256).$$

Zatem:

$$\frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = (-1, -1).$$

Przy $\eta = 0.01$:

$$w^{(1)} \approx (0.01, 0.01).$$

Adam w pierwszym kroku wykorzystuje głównie znaki gradientów. W kolejnych krokach relacja między m i v zawiera już informację z historii.

20.10. Przykład jednego parametru Adama przez dwa kroki

Niech gradienty jednego parametru wynoszą kolejno $g_1 = 4$ i $g_2 = 2$, a $\beta_1 = 0.9$, $\beta_2 = 0.999$.

Po pierwszym kroku:

$$m_1 = 0.4, \quad v_1 = 0.016,$$

$$\hat{m}_1 = 4, \quad \hat{v}_1 = 16.$$

Po drugim gradiencie:

$$m_2 = 0.9(0.4) + 0.1(2) = 0.56,$$

$$v_2 = 0.999(0.016) + 0.001(4) = 0.019984.$$

Korekty wykorzystują teraz $t = 2$:

$$\hat{m}_2 = \frac{0.56}{1 - 0.9^2} = \frac{0.56}{0.19} \approx 2.947,$$

$$\hat{v}_2 = \frac{0.019984}{1 - 0.999^2} = \frac{0.019984}{0.001999} \approx 9.997.$$

Znormalizowany kierunek ma wartość:

$$\frac{\hat{m}_2}{\sqrt{\hat{v}_2}} \approx 0.932.$$

Nie jest to po prostu aktualny gradient 2. Adam łączy historię kierunku z historią kwadratów skali.

21. Połączenie z dalszym machine learningiem

21.1. Regresja jako pojedynczy neuron liniowy

Operacja:

$$z = x^\top w + b$$

jest podstawową operacją neuronu i warstwy liniowej. W regresji liniowej wynik z jest bezpośrednio predykcją. W sieci neuronowej może zostać przekazany przez funkcję aktywacji i kolejne warstwy.

Dla batcha warstwa liniowa wykonuje dokładnie macierzowe:

$$Z = XW + b.$$

To rozszerzenie $X @ w + b$ na wiele neuronów.

21.2. d_out jako początek backpropagation

Projekt oblicza:

$$\frac{\partial J}{\partial \hat{y}} = \frac{2}{n}(\hat{y} - y).$$

Następnie przez regułę łańcuchową propaguje tę pochodną do wag i biasu. Jest to już najprostszy przypadek **backpropagation** (propagacji wstecznej). W głębszej sieci proces jest dłuższy, ale zasada pozostaje taka sama:

1. forward pass - obliczenie predykcji i loss,
2. backward pass - obliczenie gradientów od wyjścia do wcześniejszych parametrów,
3. optimizer step - aktualizacja parametrów.

W `_fit_gd` te etapy występują bez frameworka automatycznego różniczkowania.

21.3. Stan optymalizatora skaluje się do sieci

Momentum przechowuje jedną dodatkową liczbę dla każdego parametru. Adam przechowuje dwie: pierwszy i drugi moment. Dla modelu z miliardem parametrów Adam potrzebuje więc ogromnej dodatkowej pamięci, oprócz samych parametrów i gradientów.

W projekcie są to małe tablice `m_w`, `v_w` i skalary dla biasu. Mechanizm jest jednak ten sam.

21.4. Mini-batche stają się koniecznością

Dla ogromnego datasetu pełnego gradientu nie da się wygodnie policzyć w każdym kroku. Mini-batch:

- mieści się w pamięci,
- wykorzystuje równoległość GPU,
- pozwala wykonywać częste aktualizacje,
- daje zaszumiony, ale użyteczny gradient.

Twoja pętla batchowa jest uproszczonym odpowiednikiem `DataLoader` używanego np. w PyTorch.

21.5. Różnica: problem wypukły i niewypukły

Regresja liniowa z MSE jest wypukła. Głębokie sieci mają niewypukłe funkcje kosztu. W regresji pytamy głównie, jak szybko dotrzeć do jednego globalnego rozwiązania; w sieciach trajektoria optymalizatora może prowadzić do różnych rozwiązań o odmiennych właściwościach.

Dlatego intuicje z regresji są fundamentem, ale nie wszystkie twierdzenia przenoszą się bez zmian.

21.6. Regularizacja

Kara L_2 jest w sieciach często nazywana **weight decay**. Klasyczne dodanie $\alpha\|w\|^2$ do funkcji celu i implementacyjny weight decay są równoważne dla zwykłego SGD przy odpowiedniej skali, ale przy adaptacyjnych optymalizatorach, takich jak Adam, rozdzielony weight decay (np. AdamW) nie jest identyczny ze zwykłym dodaniem gradientu kary przed adaptacyjnym skalowaniem.

To jest późniejszy temat, ale projekt pokazuje początek tej idei.

22. Pytania kontrolne i eksperymenty

22.1. Pytania koncepcyjne

1. Dlaczego $X @ w$ daje jedną predykcję na obserwację?
2. Czym różni się parametr od hiperparametru?
3. Dlaczego znak błędu nie znika w gradiencie, mimo że MSE używa kwadratu?
4. Skąd w gradiencie wag bierze się X^T ?
5. Dlaczego gradient biasu jest sumą d_{out} ?
6. Co dokładnie oznacza jedna epoka dla `batch_size=32`?
7. Dlaczego t w Adamie rośnie na batch, a nie tylko raz na epokę?
8. Dlaczego pierwszy krok Momentum w tej implementacji jest mniejszy od Vanilla?
9. Po co Adamowi zarówno m , jak i v ?
10. Dlaczego korekta obciążenia jest najsilniejsza na początku treningu?
11. Dlaczego standaryzacja cech pomaga gradient descent, choć nie zmienia zasadniczej klasy zależności liniowych?
12. Jak współliniowość może zmieniać wagi bez dużej zmiany predykcji?
13. Dlaczego Ridge nie reguluje biasu?
14. Czy $R^2 < 0$ jest możliwe i co oznacza?
15. Dlaczego treningowe R^2 nie jest dowodem dobrej generalizacji?

22.2. Ćwiczenia matematyczne

1. Wyprowadź gradient dla funkcji z czynnikiem $1/(2n)$ i porównaj go z projektem.
2. Dodaj bias do wektora parametrów i wyprowadź jeden wspólny gradient względem θ .
3. Pokaż, że $X^T X$ jest dodatnio półokreślona.
4. Dla $f(x, y) = x^2 + 5y^2$ znajdź przedział learning rate zapewniający zbieżność Vanilla GD bez szumu.
5. Wyprowadź analityczne x_t i y_t dla Vanilla GD na tej funkcji.
6. Wyprowadź normal equations z warunku ortogonalności reszt do przestrzeni kolumn X .
7. Wyprowadź rozwiązanie Ridge dla scentrowanych danych.

Rozwiązanie wskazówki do ćwiczenia 4

Największa wartość własna Hessianu wynosi 10, więc:

$$0 < \eta < \frac{2}{10} = 0.2.$$

Przy $\eta = 0.2$ czynnik dla kierunku y wynosi $1 - 0.2 \cdot 10 = -1$, więc nie ma zaniku amplitudy; ścisła nierówność jest istotna.

22.3. Eksperymenty kodowe

Eksperyment A - learning rate

Uruchom full-batch GD dla:

```
0.0001, 0.001, 0.01, 0.1, 1.0
```

Porównaj loss i trajektorie wag. Przed uruchomieniem zapisz przewidywanie: wolna zbieżność, szybka zbieżność, oscylacja czy rozbieżność.

Eksperyment B - skala jednej cechy

Pomnóż jedną kolumnę X przez 100 bez zmiany $1r$. Następnie wystandaryzuj dane. Porównaj stabilność. To praktyczna demonstracja Hessianu i uwarunkowania.

Eksperyment C - batch size

Porównaj None, 64, 32, 8 i 1. Zapisuj:

- loss po każdej epoce na całym zbiorze,
- loss po każdym kroku,
- czas treningu,
- liczbę aktualizacji.

Eksperyment D - siła Ridge

Użyj α równych:

```
0, 0.001, 0.01, 0.1, 1, 10
```

Porównaj MSE, $\|w\|_2$, współczynniki dwóch współliniowych cech i jakość na danych testowych.

Eksperyment E - poziom szumu

Zmień odchylenie szumu danych z 0.5 na 0, 0.1, 1 i 3. Sprawdź rozrzut oszacowanych wag w wielu powtórzeniach. To pokazuje wariancję estymatora.

Eksperyment F - kontrola gradientu

Porównaj gradient analityczny z przybliżeniem różnicą centralną:

$$\frac{\partial J}{\partial w_j} \approx \frac{J(w + he_j) - J(w - he_j)}{2h}.$$

Dla umiarkowanego h , np. 10^{-5} , wyniki powinny być bardzo bliskie. Jest to **gradient checking** - sprawdzanie poprawności ręcznie zaimplementowanej pochodnej.

Eksperyment G - uczciwsze porównanie optimizerów

Wykonaj wiele powtórzeń, raportuj średnią oraz odchylenie wyników i dostrój osobny learning rate dla każdego optimizera. Porównaj też przypadek bez szumu. Dzięki temu oddzielisz zachowanie algorytmu od pojedynczego losowania.

22.4. Kryterium pełnego zrozumienia projektu

Projekt jest naprawdę zrozumiany, jeśli potrafisz bez zaglądania do kodu:

1. podać wymiary każdego obiektu w `_fit_gd`,
2. wyprowadzić `d_out`, `grad_w` i `grad_b`,
3. opisać różnicę między epoką a krokiem,
4. ręcznie wykonać jedną aktualizację Vanilla, Momentum i Adam,
5. wyjaśnić, dlaczego `lstsq` i GD prowadzą do podobnych wyników,
6. wyjaśnić wpływ skali cech i współliniowości,
7. powiedzieć, czego nie dowodzi treningowe R^2 ,
8. wskazać ograniczenia eksperymentu z optimizerami,
9. połączyć `d_out` z regułą łańcuchową i backpropagation.

23. Słownik pojęć

Termin	Znaczenie w tym projekcie
Machine learning (ML)	Uczenie parametrów modelu na podstawie danych zamiast ręcznego wpisania wszystkich reguł predykcji.
Supervised learning	Uczenie nadzorowane: każda treningowa obserwacja ma cechy X i znany target y .
Regression	Zadanie przewidywania wartości liczbowej.
Sample / observation	Jedna obserwacja, czyli jeden wiersz X i odpowiadająca mu wartość y .
Feature	Cecha wejściowa, czyli jedna kolumna X .
Target	Wartość, którą model ma przewidywać.
Prediction	Wynik modelu $\hat{y}$.
Residual	Różnica między wartością rzeczywistą a dopasowaną; konwencja znaku może być $y - \hat{y}$ lub odwrotna.
Weight / coefficient	Współczynnik określający wpływ cechy na liniową predykcję.
Bias / intercept	Wyraz wolny modelu; nie należy go mylić z obciążeniem statystycznym.
Parameter	Wartość uczona z danych, tutaj wagi i bias.
Hyperparameter	Ustawienie procesu uczenia, np. learning rate lub α .
Fit / training	Proces dopasowania parametrów modelu.
Inference / prediction	Użycie już dopasowanego modelu do obliczenia wyników. W statystyce słowo inference może też oznaczać wnioskowanie statystyczne.
Loss / cost / objective	Skalarna funkcja określająca, jak złe są parametry; w tym projekcie głównie MSE, opcjonalnie z karą Ridge.
MSE	Mean squared error, średni błąd kwadratowy.
Gradient	Wektor pochodnych cząstkowych funkcji skalarnej względem parametrów.
Hessian	Macierz drugich pochodnych opisująca lokalną krzywiznę.
Gradient descent	Iteracyjna minimalizacja przez kroki przeciwne do gradientu.
Learning rate	Skala kroku optymalizatora.
Optimizer	Reguła wykorzystująca gradient do aktualizacji parametrów.
Vanilla GD	Podstawowy gradient descent bez pamięci ani adaptacyjnego skalowania.
Momentum	Optymalizator używający wykładniczej średniej wcześniejszych gradientów.
Adam	Optymalizator używający pierwszego i drugiego momentu gradientów oraz adaptacyjnej skali.
Moment	W Adamie: średnia gradientów i średnia ich kwadratów; są to estymaty wygładzane wykładniczo.
Bias correction	Korekta początkowego przyciągania momentów Adama do zera; nie jest tym samym co bias regresji.
Epoch	Jedno przejście przez cały treningowy zbiór danych.
Batch	Zbiór obserwacji użyty do jednego gradientu i kroku.
Mini-batch	Batch mniejszy od całego datasetu, ale zwykle większy od jednej obserwacji.
SGD	Stochastic gradient descent; ściśle aktualizacja na jednej losowej obserwacji, szerzej także losowe mini-batche.
Convergence	Zbieżność parametrów lub wartości funkcji do granicy/optimum.
Divergence	Rozbieżność: oddalanie się od optimum, często wskutek zbyt dużego learning rate.

Termin	Znaczenie w tym projekcie
Convexity	Własność sprawiająca, że każde minimum lokalne jest globalne.
Condition number	Miara nierównomierności skali/krzywizny i wrażliwości numerycznej problemu.
Standardization	Odejęcie średniej i podzielenie przez odchylenie dla każdej cechy.
Closed-form solution	Rozwiązanie parametrów bez użytkowej pętli gradient descent, tu przez least squares.
Least squares / OLS	Metoda minimalizująca sumę kwadratów reszt.
Normal equations	Warunek $X^T X w = X^T y$ wynikający z zerowego gradientu.
Pseudoinverse	Uogólnienie macierzy odwrotnej użyteczne dla macierzy prostokątnych lub bez pełnej rangi.
SVD	Rozkład macierzy według wartości osobliwych, użyteczny w stabilnym least squares.
Multicollinearity	Silna liniowa zależność pomiędzy cechami.
Regularization	Modyfikacja celu ograniczająca złożoność lub wielkość parametrów.
Ridge / L2 regularization	Kara proporcjonalna do sumy kwadratów wag.
Lasso / L1 regularization	Kara proporcjonalna do sumy wartości bezwzględnych wag.
Generalization	Zdolność modelu do działania na nowych danych.
Overfitting	Nadmierne dopasowanie do train kosztem jakości na nowych danych.
Data leakage	Użycie informacji spoza danych treningowych w procesie uczenia lub przygotowania modelu.
Train/test split	Podział danych na część do uczenia i niezależną część do oceny.
R^2	Współczynnik determinacji porównujący SSE modelu z predykcją średniej.
Vectorization	Zapis obliczeń jako operacji na całych tablicach zamiast pętli po elementach.
Broadcasting	Reguły NumPy pozwalające wykonywać działania na zgodnych, lecz różnych kształtach tablic.
Forward pass	Obliczenie predykcji i funkcji kosztu od wejścia do wyjścia.
Backward pass	Obliczanie gradientów od funkcji kosztu wstecz do parametrów.
Backpropagation	Efektywne zastosowanie reguły łańcuchowej przez graf obliczeń.

Aneks A. Niezmieniony kod analizowany w dokumencie

Poniższe listingi są kopią analizowanej wersji. Nie są „poprawioną” implementacją. Ich celem jest uczynienie dokumentu samowystarczalnym: opis matematyczny, komentarze i dokładny kod znajdujący się w jednym pliku.

A.1. metrics.py

```
import numpy as np

def r_squared(y_true: np.ndarray, y_pred: np.ndarray) -> float:

    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - y_true.mean()) ** 2)
```

```

    return 1 - ss_res / ss_tot

def mse(y_true: np.ndarray, y_pred: np.ndarray) -> float:

    return np.mean((y_true - y_pred) ** 2)

```

Mapa omówienia:

- import i operacje NumPy - rozdział 17,
- r_squared - rozdział 14.2-14.4,
- mse - rozdział 5 oraz 14.1.

A.2. optimizers.py

```

import numpy as np

def vanilla_step(params: np.ndarray, grad: np.ndarray, lr: float) -> np.ndarray:
    return params - grad * lr

def momentum_step(
    params: np.ndarray,
    grad: np.ndarray,
    velocity: np.ndarray,
    lr: float,
    beta: float = 0.9,
) -> tuple[np.ndarray, np.ndarray]:
    velocity = beta * velocity + (1 - beta) * grad
    params = params - lr * velocity
    return params, velocity

def adam_step(
    params: np.ndarray,
    grad: np.ndarray,
    m: np.ndarray,
    v: np.ndarray,
    t: int,
    lr: float,
    beta1: float = 0.9,
    beta2: float = 0.999,
    eps: float = 1e-8,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    m = beta1 * m + (1 - beta1) * grad
    v = beta2 * v + (1 - beta2) * grad**2
    m_hat = m / (1 - beta1**t)
    v_hat = v / (1 - beta2**t)
    params = params - lr * m_hat / (np.sqrt(v_hat) + eps)
    return params, m, v

```

Mapa omówienia:

- sens wspólnego interfejsu - rozdział 12.1,
- vanilla_step - rozdział 12.2,
- momentum_step - rozdział 12.3,
- adam_step - rozdział 12.4,

- typy zwracanych krotek i rozpakowanie - rozdziały 12.5 i 17.6.

A.3. model.py

```
import numpy as np
from .metrics import r_squared, mse
from .optimizers import vanilla_step, momentum_step, adam_step

class LinearRegression:
    def __init__(
        self,
        method: str = "gd",
        lr: float = 0.01,
        n_epochs: int = 100,
        batch_size: int | None = None,
        alpha: float = 0.0,
        optimizer: str = "vanilla",
        seed: int = 42,
    ) -> None:
        self.method = method
        self.lr = lr
        self.n_epochs = n_epochs
        self.batch_size = batch_size
        self.alpha = alpha
        self.optimizer = optimizer
        self.seed = seed
        self.w = None
        self.b = None
        self.loss_history = []
        self.weight_history = []

    def fit(self, X: np.ndarray, y: np.ndarray) -> "LinearRegression":
        if self.method == "gd":
            self._fit_gd(X, y)
        elif self.method == "closed_form":
            self._fit_closed_form(X, y)
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        return X @ self.w + self.b

    def score(self, X: np.ndarray, y: np.ndarray) -> float:
        y_pred = self.predict(X)
        return r_squared(y, y_pred)

    def _fit_closed_form(self, X: np.ndarray, y: np.ndarray) -> None:
        ones = np.ones((X.shape[0], 1))
        X_aug = np.hstack([X, ones])
        coef = np.linalg.lstsq(X_aug, y, rcond=None)[0]
        self.w = coef[:-1]
        self.b = coef[-1]

    def _fit_gd(self, X: np.ndarray, y: np.ndarray) -> None:
        t = 0
        self.b = 0.0
        velocity_b = 0.0
        v_b = 0.0
        m_b = 0.0
        self.w = np.zeros(X.shape[1])
```

```

velocity_w = np.zeros(X.shape[1])
v_w = np.zeros(X.shape[1])
m_w = np.zeros(X.shape[1])

for _ in range(self.n_epochs):
    batch_loss = []
    batch_size = self.batch_size if self.batch_size is not None else X.shape[0]
    for batch in range(0, X.shape[0], batch_size):
        X_batch = X[batch : batch + batch_size]
        y_batch = y[batch : batch + batch_size]
        y_pred = X_batch @ self.w + self.b
        d_out = 2 / y_batch.size * (y_pred - y_batch)
        grad_w = X_batch.T @ d_out
        grad_w = grad_w + 2 * self.alpha * self.w
        grad_b = d_out.sum()

        if self.optimizer == "vanilla":
            self.w = vanilla_step(self.w, grad_w, self.lr)
            self.b = vanilla_step(self.b, grad_b, self.lr)
        elif self.optimizer == "momentum":
            self.w, velocity_w = momentum_step(
                self.w, grad_w, velocity_w, self.lr
            )
            self.b, velocity_b = momentum_step(
                self.b, grad_b, velocity_b, self.lr
            )
        elif self.optimizer == "adam":
            t += 1
            self.w, m_w, v_w = adam_step(self.w, grad_w, m_w, v_w, t, self.lr)
            self.b, m_b, v_b = adam_step(self.b, grad_b, m_b, v_b, t, self.lr)
        batch_loss.append(mse(y_batch, y_pred))
    self.loss_history.append(np.mean(batch_loss))
    self.weight_history.append(self.w.copy())

```

Mapa omówienia:

- importy i struktura pakietu - rozdział 17,
- konstruktor, pola i konfiguracja - rozdział 11.1,
- fit, predict, score - rozdziały 11.2-11.4,
- _fit_closed_form - rozdział 8,
- inicjalizacja _fit_gd - rozdziały 9.3 i 11.5,
- pętle epok i batchy - rozdziały 9.4, 10 i 11.6,
- predykcja i gradienty - rozdziały 3, 5 i 6,
- regularizacja - rozdział 13,
- wybór optymalizatora - rozdział 12,
- historie - rozdziały 10.6 i 11.7.

A.4. run_experiments.py

```

import numpy as np
from .model import LinearRegression
from sklearn.datasets import fetch_california_housing
from .optimizers import vanilla_step, momentum_step, adam_step
from .visualize import (
    plot_loss_curves,
    plot_predictions_vs_truth,
    plot_weight_convergence,
    plot_optimizer_comparison,
)

```

```

def main() -> None:
    X = np.random.randn(200, 3)
    true_w = np.array([2.5, -1.3, 0.7])
    true_b = 4.0
    noise = np.random.randn(200) * 0.5
    y = X @ true_w + true_b + noise

    model_cf = LinearRegression(method="closed_form").fit(X, y)
    assert np.allclose(true_w, model_cf.w, atol=0.1)
    assert np.isclose(true_b, model_cf.b, atol=0.1)

    model_gd = LinearRegression(method="gd", n_epochs=1000).fit(X, y)
    assert np.allclose(true_w, model_gd.w, atol=0.1)
    assert np.isclose(true_b, model_gd.b, atol=0.1)

    batch_gd = LinearRegression(method="gd").fit(X, y)
    mini_batch_gd = LinearRegression(method="gd", batch_size=32).fit(X, y)
    plot_loss_curves(batch_gd.loss_history, mini_batch_gd.loss_history)

    X_collinear = X.copy()
    X_collinear[:, 0] = X_collinear[:, 0] + np.random.randn(200) * 0.01
    X_extended = np.hstack([X, X_collinear[:, 0]])
    model_ridge = LinearRegression(method="gd", alpha=0.1, n_epochs=1000).fit(
        X_extended, y
    )
    model_plain = LinearRegression(method="gd", alpha=0.0, n_epochs=1000).fit(
        X_extended, y
    )
    assert np.abs(model_ridge.w).sum() < np.abs(model_plain.w).sum()

    plot_weight_convergence(np.array(model_gd.weight_history))

    data = fetch_california_housing()
    X, y = data.data, data.target
    X_scaled = (X - X.mean(axis=0)) / X.std(axis=0)
    model_housing = LinearRegression(method="gd", n_epochs=1000).fit(X_scaled, y)
    r2 = model_housing.score(X_scaled, y)
    y_pred = model_housing.predict(X_scaled)

    assert 0.5 < r2 < 0.7
    plot_predictions_vs_truth(y, y_pred)

    def toy_function(params: np.ndarray) -> float:
        x, y = params[0], params[1]
        return x**2 + 5 * y**2

    def toy_gradient(params: np.ndarray, noise_std: float = 2.0) -> np.ndarray:
        x, y = params[0], params[1]
        true_grad = np.array([2 * x, 10 * y])
        return true_grad + np.random.randn(2) * noise_std

    params = np.array([3.0, 3.0])
    vanilla_losses = []

    for _ in range(100):
        grad = toy_gradient(params)
        params = vanilla_step(params, grad, lr=0.02)
        vanilla_losses.append(toy_function(params))

```

```

params = np.array([3.0, 3.0])
velocity = np.zeros(2)
momentum_losses = []

for _ in range(100):
    grad = toy_gradient(params)
    params, velocity = momentum_step(params, grad, velocity, lr=0.02)
    momentum_losses.append(toy_function(params))

params = np.array([3.0, 3.0])
m = np.zeros(2)
v = np.zeros(2)
t = 0
adam_losses = []

for _ in range(100):
    grad = toy_gradient(params)
    t += 1
    params, m, v = adam_step(params, grad, m, v, t, lr=0.02)
    adam_losses.append(toy_function(params))

optimizers = {
    "vanilla": vanilla_losses,
    "momentum": momentum_losses,
    "adam": adam_losses,
}
plot_optimizer_comparison(optimizers)

# with a shared learning rate tuned for vanilla GD, Adam converges far slower on this
↪ simple,
# low-noise-relative-to-signal toy problem, since it's normalized per-parameter step size
# doesn't exploit the large early gradient magnitude the way vanilla/momentum do.
# Adam's advantage shows up on noisier or messier loss surfaces, not here.

if __name__ == "__main__":
    main()

```

Mapa omówienia:

- dane syntetyczne - rozdział 4,
- asercje modeli - rozdziały 15.2-15.3,
- batch kontra mini-batch - rozdziały 10 i 15.4,
- współliniowość i Ridge - rozdziały 13 i 15.5,
- historia wag - rozdział 15.6,
- California Housing - rozdział 15.7,
- funkcja kwadratowa i szum - rozdziały 15.8-15.13,
- `__main__` - rozdział 15.14.

W drugim wierszu komentarza jest drobna literówka językowa: `it's normalized` powinno być `its normalized`. Nie zmienia ona działania programu.

A.5. visualize.py

```

import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

```

```

FIGURES_DIR = Path(__file__).parent / "figures"

def plot_loss_curves(batch_losses: list[float], minibatch_losses: list[float]) -> None:
    FIGURES_DIR.mkdir(exist_ok=True)
    fig, ax = plt.subplots()
    ax.plot(batch_losses, label="batch GD")
    ax.plot(minibatch_losses, label="mini-batch GD")
    ax.set_xlabel("epoch")
    ax.set_ylabel("loss (MSE)")
    ax.set_title("Batch vs Mini-batch GD - Loss curves")
    ax.legend()
    fig.savefig(FIGURES_DIR / "loss_curves.png")

def plot_predictions_vs_truth(y_true: np.ndarray, y_pred: np.ndarray) -> None:
    FIGURES_DIR.mkdir(exist_ok=True)
    fig, ax = plt.subplots()
    ax.scatter(y_true, y_pred)
    lo = min(y_true.min(), y_pred.min())
    hi = max(y_true.max(), y_pred.max())
    ax.axline([lo, lo], [hi, hi])
    ax.set_xlabel("true values")
    ax.set_ylabel("predicted values")
    ax.set_title("True vs Predicted values")
    fig.savefig(FIGURES_DIR / "predictions.png")

def plot_weight_convergence(weight_history: np.ndarray) -> None:
    FIGURES_DIR.mkdir(exist_ok=True)
    fig, ax = plt.subplots()
    for i in range(weight_history.shape[1]):
        ax.plot(weight_history[:, i], label=f"w{i}")
    ax.set_xlabel("epoch")
    ax.set_ylabel("weight")
    ax.set_title("Weight convergence over epochs")
    ax.legend()
    fig.savefig(FIGURES_DIR / "weight_convergence.png")

def plot_optimizer_comparison(loss_histories: dict[str, list[float]]) -> None:
    FIGURES_DIR.mkdir(exist_ok=True)
    fig, ax = plt.subplots()
    for label, loss_history in loss_histories.items():
        ax.plot(loss_history, label=label)
    ax.set_xlabel("step")
    ax.set_ylabel("loss")
    ax.set_title("Optimizer loss comparison")
    ax.legend()
    fig.savefig(FIGURES_DIR / "optimizer_comparison.png")

```

Wszystkie linie tego pliku zostały omówione w rozdziale 16, a związane z nimi operacje NumPy i typowanie również w rozdziale 17.

A.6. __init__.py

Plik jest pusty. Jego rola została wyjaśniona w rozdziale 17.1.

Końcowa synteza

Projekt implementuje dwa sposoby rozwiązania jednego podstawowego problemu:

$$\min_{w,b} \frac{1}{n} \|Xw + b\mathbf{1} - y\|_2^2.$$

Rozwiązanie `closed_form` znajduje minimum przez numeryczne least squares. Ścieżka `gd` zaczyna od zerowych parametrów, liczy predykcje, pochodną loss względem predykcji, gradient wag i biasu, a następnie aktualizuje parametry jednym z trzech optimizerów.

Najważniejsza zależność w całym kodzie to:

$$\hat{y} = Xw + b \implies \frac{\partial J}{\partial \hat{y}} = \frac{2}{n}(\hat{y} - y) \implies \nabla_w J = X^\top \frac{\partial J}{\partial \hat{y}}$$

To jest jednocześnie kompletna matematyka gradientu regresji liniowej i najprostszy przykład mechanizmu, który później staje się backpropagation w sieci neuronowej.

Ridge dodaje do tego gradient $2\alpha w$, Momentum wygładza gradienty w czasie, a Adam dodatkowo normalizuje je przez estymowaną skalę kwadratów gradientów. Batch size decyduje, na ilu obserwacjach powstaje pojedynczy gradient, zaś learning rate określa, jak silnie optimizer reaguje na uzyskany kierunek.

Kod ma kilka uproszczeń w walidacji, monitorowaniu i konstrukcji eksperymentów, ale jego główny rdzeń matematyczny jest poprawny. Najważniejsze ograniczenia nie polegają na tym, że model „nie działa”, lecz na tym, że nie każdy wykres i każda asercja uzasadniają tak szerokie wnioski, jak mogłoby się początkowo wydawać.

Po opanowaniu tego materiału kolejnym naturalnym krokiem jest implementacja regresji w PyTorch i porównanie ręcznie policzonych gradientów z `autograd`, a następnie regresja logistyczna, pojedyncza warstwa neuronowa i pełny backpropagation.